Reinventing Multi-User Authentication Security from Cross-Chain Perspective

Yongyang Lv, Ruitao Feng*, Maode Ma, *Senior Member, IEEE*,
Manqing Zhu, Hanwei Wu, Xiaohong Li*, *Member, IEEE*

***Abstract*—Blockchain systems encompass many distinct and autonomous entities, each utilizing its own self-contained identity authentication algorithm. Unlike identity authentication within a singular blockchain, cross-chain scenarios demand special attention due to their pivotal role in enabling the acknowledgment of users' identities across diverse domains. This capability is the foundational prerequisite for the circulation of resources across different chains. Consequently, the central challenge for cross-chain systems lies in establishing mutual recognition and trust in users' digital identities.**

This paper proposes a Multi-User Proxy Re-Signature (MU-PRS) algorithm, facilitating the cross-chain conversion of signatures from multiple users. Concurrently, This paper propose the Multi-Notary Signature Conversion (MN-SC) mechanism, designed to address the challenge posed by disparate system mechanisms across blockchains during cross-chain authentication. Leveraging the MU-PRS algorithm and MN-SC mechanism, we present a Multi-User Cross-Chain Authentication Scheme (MU-CCAS) within a heterogeneous blockchain environment. This scheme enables the verification of identities of multiple cross-chain users through a single signature verification. This innovative approach not only addresses the centralization issues inherent in third-party cross-chain authentication but also significantly enhances the efficiency of identity authentication. The evaluation results demonstrate MU-CCAS's superior security over existing solutions in three dimensions: BAN logic, *Scyther* verification, and security attribute analysis. Additionally, it establishes that MU-PRS and MU-CCAS have low computational overhead, easy implementation, and excel in algorithm, scheme, and cross-chain performance. Overall, our work provides a robust and efficient framework for cross-chain authentication, addressing centralization challenges and enhancing digital security.

***Index Terms*—Blockchain; Cross-Chain Authentication; Heterogeneous; Proxy Re-Signature**

I. Introduction

SINCE its inception, blockchain technology [1] has found extensive applications in various fields such as finance [2], [3] and healthcare [4], [5]. However, many existing blockchain systems operate as independent value ecosystems, making it challenging for data to flow between different blockchain systems [6], [7]. Cross-chain authentication has emerged as a research hotspot because it enables asset transfer and resource transfers between different blockchain systems [8], [9]. In the context of consortium blockchains, achieving resource transfer among different blockchain systems requires cross-chain authentication [10], which involves identifying users across different systems to ensure the security of data and user identity.

Leveraging blockchain's authentication capabilities, numerous studies view blockchain as a distributed trust entity, applicable to various trust system resource transfer scenarios [11], [12]. This is accomplished by establishing trust roots [13], [14] and employing signature validation mechanisms [15], [16], [17], [18]. However, these schemes typically confine themselves to a single blockchain environment, termed cross-domain authentication. It is crucial to distinguish between cross-chain and cross-domain authentication: cross-domain authentication examines trust domains within the same blockchain [19], whereas cross-chain authentication addresses not only trust domain issues but also the challenges posed by disparate blockchain system mechanisms (i.e., cross-chain heterogeneous environment) [6]. Existing cross-chain authentication schemes [2], [3], [20], [21], [22] predominantly concentrate on the transfer and exchange of digital assets, with the primary goal of addressing the atomicity issue within cross-chain mechanisms. However, these schemes are not well-suited for facilitating cross-chain resource transfers [23]. We summarize the problems solved by the above schemes in Table I.

TABLE I: Comparison of problems solved by existing work

Scheme	Heterogeneous environment	Decentralization		Multi-user identity authentication
		Cross-chain	Cross-domain	
Zhang et al. [2]	No	Yes	No	No
Chen et al. [3]	No	Yes	No	No
Yang et al. [13]	No	No	Yes	No
Lai et al. [14]	No	No	Yes	No
Dong et al. [15]	No	No	Yes	No
Zhang et al. [16]	Yes	No	Yes	No
Wang et al. [17]	No	No	Yes	No
Wang et al. [18]	No	No	Yes	Yes
Sun et al. [20]	No	Yes	No	No
Garoffolo et al. [21]	No	Yes	No	No
Yin et al. [22]	No	Yes	No	No
Ours	Yes	Yes	Yes	Yes

In the context of cross-chain authentication for resource transfer, there are three critical challenges that require further investigation. In terms of **functionality**, it is currently infeasible to authenticate user identities across different heterogeneous blockchains [24]. In terms of **security**, the use of centralized inter-chain nodes for auxiliary verification during cross-chain authentication introduces the risk of a single point of failure [25]. In terms of **performance**, the existing solution [23] is limited to one-to-one user authentication, which is

*Corresponding author

Y. Lv, M. Zhu, H. Wu, X. Li are with the College of Intelligence and Computing, Tianjin University, China.
E-mail:{lvyongyang, manqingzhu, wuhanwei, xiaohongli}@tju.edu.cn

R. Feng is with the School of Computing and Information Systems, Singapore Management University, Singapore.
E-mail: rtfeng@smu.edu.sg

M. Ma is with the College of Engineering, Qatar University, Qatar.
E-mail: mammdsg@ieee.org

inefficient. Consequently, there is a pressing need to develop a secure and efficient cross-chain authentication scheme.

To illustrate the limitations of previous methods, let's consider a toy example in Figure 1, imagine three user needing to access resources from a database DB across the chain. For instance, as depicted in part (a), the previous scheme encounters three primary issues. Firstly, the authentication scheme operates within a homogeneous environment, rendering it of limited practical utility. Secondly, relying on a single notary for identity verification introduces numerous security vulnerabilities. Thirdly, simultaneous access by multiple users necessitates individual verification, delaying the process of identifying all users' identities. Addressing these concerns forms the core objective of our paper. Specifically, we aim to develop a cross-chain authentication scheme as shown in part (b) to: 1) Address the challenges of cross-chain authentication in a **heterogeneous environment**. 2) Tackles the **centralization problem** of a single notary in cross-chain scenarios, enhancing the reliability and security of the cross-chain authentication scheme. 3) Achieves simultaneous **authentication of multiple cross-chain user identities** to cater to scenarios involving multiple users with joint authorization or shared access in cross-chain applications.

Proxy Re-Signature (PRS) enables the conversion of users' signatures across different systems via intermediaries, allowing for the verification of cross-system users' identities through the verification of transformed signatures alone. As such, PRS has become a practical algorithm for cross-domain authentication [26]. Distinguished from other approaches [24], [27], PRS offers enhanced security in cross-system applications and necessitates fewer interactive rounds to acquire identity permissions, making it a more efficient solution for cross-system authentication challenges.

Based on this foundation, we introduce a Multi-User Proxy Re-Signature (MU-PRS) algorithm that facilitates the conversion of signatures among multiple users across disparate cryptographic systems, a task delegated to an Agent. To address the challenges posed by cross-chain systems, we develop a Multi-Notary Signature Conversion (MU-SC) mechanism, wherein multiple notaries serve as agents to resolve these issues. This mechanism ensures that only a requisite number of notaries are required to perform the signature conversion online, thereby mitigating the risk of single-point failure inherent in a single notary system. Leveraging the MU-PRS algorithm and the MU-SC mechanism, we devise a Multi-User Cross-Chain Authentication Scheme (MU-CCAS) tailored for consortium blockchains in heterogeneous environments. This scheme enables the authentication of multiple cross-chain users' identities through a single verification of the transformed signatures. Finally, we employ BAN logic, *Scyther* tool, and security attribute analysis to substantiate the security of the MU-CCAS. BAN logic is utilized to demonstrate the logical correctness of the scheme, while the formal tool *Scyther* is employed to analyze and verify attributes such as confidentiality, authentication, and integrity. The analysis of security attributes further elucidates the MU-CCAS's resilience against specific security attacks. We also show that MU-PRS and MU-CCAS have low computational overhead in terms of algorithm and scheme performance. The primary contributions of this paper are as follows:

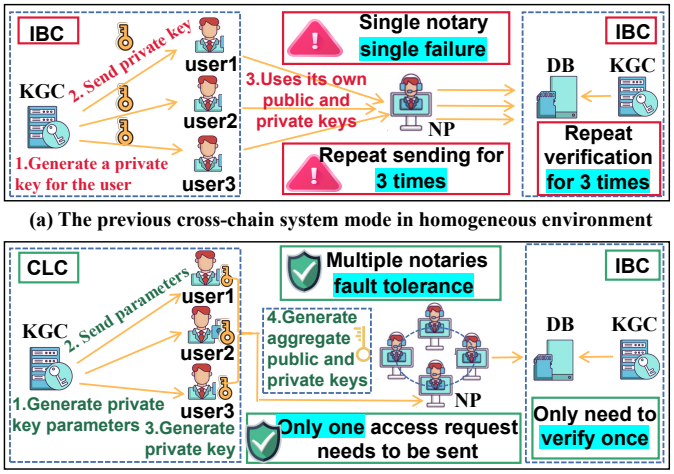


Fig. 1: A brief description of a cross-chain authentication scheme model.

1) We develop the MU-CCAS within a heterogeneous environment, providing a comprehensive description and implementing cross-chain authentication.
2) We elaborate on the MU-PRS algorithm, which is capable of simultaneously verifying signatures from multiple users across disparate cryptographic systems.
3) We detail the MN-SC mechanism, which facilitates cross-chain user identity conversion and exhibits high fault tolerance among multiple notarization nodes.
4) We demonstrate that MU-CCAS possesses robust security and performance, enhancing its practical feasibility.

This paper is structured as follows: Section II introduces the preliminary knowledge. Section III offers a detailed presentation of MU-CCAS, including its specifics. Section IV conducts a security analysis. Section V delves into a performance analysis. Section VI discusses related work in the field. Section VII engages in discussions and outlines future work. Section VIII provides a comprehensive summary of the entire paper.

II. Preliminary Knowledge

A. Bilinear Mapping

In [28], suppose G_1 and G_2 are cyclic group of prime order p, g is a generator of G_1. Define bilinear mapping e: $\mathsf{G}_1 \times \mathsf{G}_1 \rightarrow \mathsf{G}_2$. It possesses the following properties:

(1) *Bilinear*: For $\forall \mathsf{g}_1 \in \mathsf{G}_1, \forall \mathsf{g}_2 \in \mathsf{G}_1, \quad \forall \alpha, \quad \beta \in \mathsf{Z}_\mathsf{p}$, then $\mathsf{e}(\mathsf{g}_1^\alpha, \mathsf{g}_2^\beta) = \mathsf{e}(\mathsf{g}_1, \mathsf{g}_2)^{\alpha\beta}$ is established;

(2) *Non-degeneracy*: $\exists \mathsf{g}' \in \mathsf{G}_1$, then $\mathsf{e}(\mathsf{g}, \mathsf{g}) \neq 1$;

(3) *Calculability*: If '$\forall \mathsf{g}_1 \in \mathsf{G}_1$, $\forall \mathsf{g}_2 \in \mathsf{G}_1$, then $\mathsf{e}(\mathsf{g}_1, \mathsf{g}_2)$ is calculable in polynomial time.

B. Security Assumption

The security of the proposed scheme in this paper is based on the Decisional Bilinear Diffie-Hellman (DBDH) assumption [29], defined as follows:

Let e: $G_1 \times G_1 \to G_2$ be an efficiently computable bilinear map, where g is a generator of G_1. For any given $g^c \in G_1$, g, g^a, g^b, $h \in G_2$, and arbitrary g, $g^a, g^b, g^c \in G_1$, $h \in G_2$, it is hard to determine whether $h = e(g,g)^{abc}$. In other words, (g, g^a, g^b, g^c, h) cannot be distinguished with non-negligible advantage by any probabilistic polynomial-time algorithm.

C. BAN Logic

BAN logic is a modal logic grounded in belief, employed to formally assess the security of authentication protocols. In Section IV-A, this paper employs BAN logic to assess the security of the MU-CCAS. The fundamental principle of BAN logic involves deducing the final beliefs of the protocol participants from their initial beliefs through a series of logical inference rules. To gain a comprehensive understanding of these inference rules, please consult reference [30]. To evaluate an authentication protocol using BAN logic, one must initially idealize the protocol, transforming its messages into BAN logic formulae. Subsequently, based on the specific context, appropriate assumptions are made, and through the application of logical reasoning rules, it is determined whether the protocol fulfills its intended objectives. If, by the conclusion of the protocol's execution, trust in the shared communication key and the identity of the communicating party is established, this signifies the protocol's security. The specific BAN logic rules employed in this paper are delineated as follows:

(1) Rule 1 of BAN logic: The message-meaning rule

$$\frac{P| \equiv \stackrel{K}{\to} Q, \quad P \triangleleft \{X\}_{K^{-1}}}{P| \equiv Q| \sim X} \tag{1}$$

If P believes that Q possesses public key K, and P receives a message $\{X\}_{K^{-1}}$, encrypted with K^{-1} 's public key, then P believes that Q has previously sent the message X.

(2) Rule 2 of BAN logic: The nonce-verification rule

$$\frac{P| \equiv \#(X), P| \equiv Q| \sim X}{P| \equiv Q| \equiv X} \tag{2}$$

If P believes that message X is fresh and P believes that Q previously sent message X, then P believes that Q trusts message X.

(3) Rule 3 of BAN logic: The jurisdiction rule

$$\frac{P| \equiv Q| \Rightarrow X, P| \equiv Q| \equiv X}{P| \equiv X} \tag{3}$$

If P believes that Q has control over message X and P believes that Q trusts message X, then P believes message X.

(4) Rule 4 of BAN logic: The freshness rule

$$\frac{P| \equiv \#(X)}{P| \equiv \#(X,Y)} \tag{4}$$

If P believes that message X is fresh, then P believes that message (X,Y) is also fresh.

(5) Rule 5 of BAN logic: The belief rule

$$\frac{P| \equiv Q| \equiv (X,Y)}{P| \equiv Q| \equiv X} \tag{5}$$

TABLE II: Description of symbols

Symbol	Description
A_i	A user in Hospital A
DB	A database in Hospital B
NP_i	Notary public i between blockchains
KGC_A	Key generation center of Hospital A
KGC_B	Key generation center of Hospital B
KGC_C	Key generation center of NP_i
PK_A	The public key of KGC_A
PK_B	The public key of KGC_b
PK_C	The public key of NP_i
sk_{A_i}	The private key of A_i
sk_{DB}	The private key of DB
pk_{A_i}	The public key of A_i
pk_{DB}	The public key of DB
pk_{NP_i}	The public key of NP_i
SK	Aggregated private key of users
PK	Aggregated public key of users
$\delta_{A \to A_i}$	The signature of KGC_A to TID_{A_i}
$\delta_{B \to DB}$	The signature of KGC_B to TID_{DB}
R_{AB}	Re-signing key
δ_A	Aggregated signature of i users
δ_{DB}	The signature of DB

III. The MU-CCAS Scheme Based on MU-CL-PRE Algorithm

In this section, § III-A introduces the Multi-User Cross-Chain Authentication Scheme (MU-CCAS) model and its workflow. § III-B introduces the security goals of the MU-CCAS. § III-C provides a detailed explanation of the technical key points of traditional authentication schemes. § III-D analyzes the risks and efficiency in the context of cross-chain scenarios. § III-E presents an introduction to the Multi-User Proxy Re-Signature (MU-PRS) algorithm. In § III-F, the Multi-Notary Signature Conversion (MN-SC) mechanism is introduced. Finally, in § III-G, a detailed description of MU-CCAS is provided. The used symbols used are explained in Table II.

A. Overview

1) Model Definition: Based on the assumption that users initiate cross-chain accesses in two blockchain systems, $Blockchain_A$ and $Blockchain_B$, we have designed an example in a medical scenario to demonstrate the MU-CCAS model. Hospital A operates as a node within $Blockchain_A$ and employs a Certificateless Cryptography (CLC) system. Hospital B, situated in $Blockchain_B$, uses an Identity-Based Cryptography (IBC) system. The MU-CCAS model consists of five entities illustrated in Figure 2.

(a) *Blockchain* ($Blockchain_i$): It generates system-wide public parameters with multiple nodes, each containing various users or databases.

(b) *Users* (A_i): With four designated roles, Dean A_1, Attending Doctor A_2, Patient A_3, and Department Leader A_4, each user can apply for accessing data resources within or outside of his blockchain.

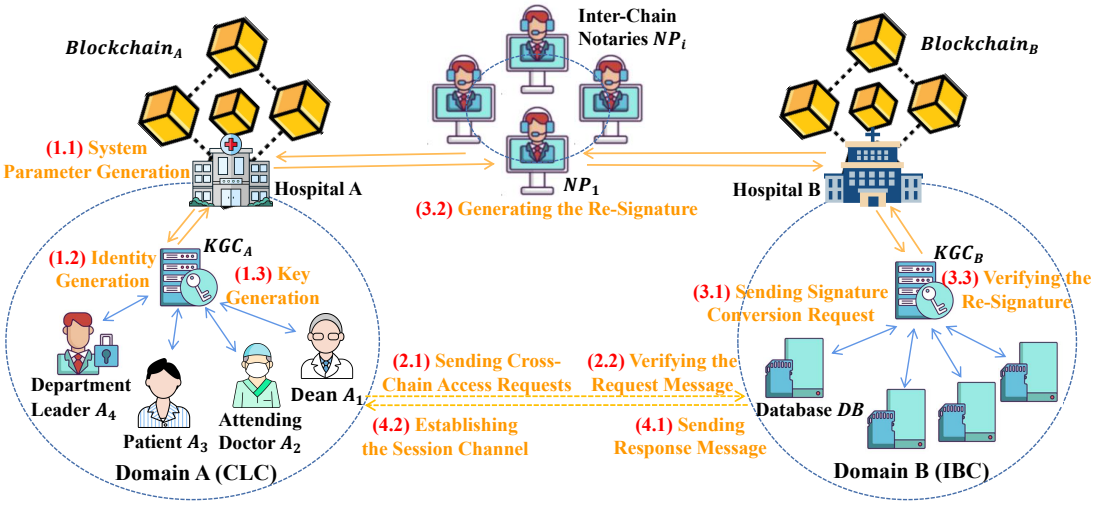


Fig. 2: MU-CCAS model

(c) *Database* (DB): It authenticates data requesters and provides data resource services.
(d) *Notary Public* ($\mathsf{NP_i}$): Provides signature conversion services for users or databases across different blockchains.
(e) *Key Generation Center* ($\mathsf{KGC_i}$): It generates identities and keys for users or databases.

2) System Workflow: Assuming that $\mathsf{A_3}$ seeks medical treatment at Hospital A. The $\mathsf{A_1}$, $\mathsf{A_2}$, $\mathsf{A_3}$ and $\mathsf{A_4}$ in Hospital A need to access $\mathsf{A_3}$'s medical record from Hospital B. The workflow of MU-CCAS in this scenario consists of the following four stages.

a) System Initialization: In the first stage, the system will be initialized in three steps.

(1.1) *System Parameter Generation*: The blockchain employ a System Initialization algorithm (***Setup***) of MU-PRS (as described in § III-E) to generate system parameters.
(1.2) *Identity Generation*: $\mathsf{KGC_i}$ generates the real identities of $\mathsf{A_i}$, DB, and $\mathsf{NP_i}$.
(1.3) *Key Generation*: KGC uses Key Generation algorithm (***KeyGen***) to generate public and private keys for $\mathsf{KGC_i}$, $\mathsf{A_i}$, and DB.

b) Multi-User Cross-Chain Access: In the second stage, multi-users will complete sending cross-chain access requests in two steps.

(2.1) *Sending Cross-Chain Access Requests*: Within Hospital A, the four users $\mathsf{A_i}$ employ the Signature Generational algorithm (***Sign***) to individually sign the request message m. Subsequently, $\mathsf{A_1}$ consolidates these four signatures into an aggregate signature denoted as δ_{A}. This aggregate signature δ_{A} and the request message m are then transmitted to Hospital B's database DB. By other schemes [23], where DB would necessitate repetitive verification of received messages, our approach requires verification only once. This streamlined process enhances verification efficiency by confirming the identities of multiple users in a single verification step.
(2.2) *Verifying the Request Message*: DB assesses whether the user initiates cross-chain access for the first time. If affirmative, it proceeds to the subsequent step. Otherwise, the user's information is already in the database, DB directly executes the repetitive cross-chain authentication operation.

c) Multi-Notary Signature Conversion: In the third stage, the multi-notary will complete the signature conversion in three steps.

(3.1) *Sending Signature Conversion Request*: The DB sends a signature conversion request to $\mathsf{NP_1}$.
(3.2) *Generating the Re-Signature*: Upon receiving the signature conversion request, the $\mathsf{NP_1}$ uses the Re-signature Key Generation algorithm (***Re-KeyGen***) and the MN-SC mechanism in § III-F to generate the re-signature key. Then $\mathsf{NP_1}$ perform the Re-signature Generation algorithm (***Re-Sign***) to convert the aggregate signature δ_{A} into a signature δ_{DB} that can be authenticated by DB. This solution solves the issue of single-point failures found in other solutions [26].
(3.3) *Verifying the Re-Signature*: The DB uses the Signature Verification algorithm (***Verify***) to verify the correctness of the signature δ_{DB}, and if the signature is correct, save the relevant information in the user access list.

d) Establishing a Session Channel: In the fourth stage, $\mathsf{A_1}$ and DB will establish a session channel in two steps.

(4.1) *Sending Response Message*: DB calculates the session key and sends a response message containing the session key to $\mathsf{A_1}$.
(4.2) *Establishing the Session Channel*: The $\mathsf{A_1}$ verifies the response message, computes the session key, and facilitates the establishment of a session channel between Hospitals A and B. It allows the authenticated users $\mathsf{A_i}$ from Hospital A to access the resources of Hospital B's database DB.

More details of MU-CCAS will be introduced in § III-G.

B. Security Goals

1) Correctness: Ensuring the accurate execution of protocols in alignment with established rules and standards. (The proof in § IV-A)

2) Confidentiality: Users' keys, data and other information should be protected from enemy attacks. (The proof in § IV-A and § IV-B)

3) Integrity: It can prove that the message content has not been modified during transmission. (The proof in § IV-A and § IV-B)

4) Authentication: Both the sender and receiver of information are capable of authenticating their identities and ensuring the secure transmission of data. (The proof in § IV-B)

C. Baselines

1) Key Generation Based on IBC and CLC: IBC [31] allows any public string to be used as a device's identifier. IBC relies on a trusted KGC to generate a private key corresponding to the device identifier. In CLC [32] , the user's public key is generated by the KGC, while the private key is co-generated by the KGC and the user. Different cryptographic systems cannot decrypt each other's encrypted messages.

2) Authentication Mechanism Based on a Single Notary: In the situations when party A needs to access the resources of party B, a notary is required to convert the signature δ_{A} provided by A in the request message m into a signature δ_{b} for the message m, thereby achieving the authentication of A's identity. This mechanism facilitates secure authentication between parties A and B, with the notary serving as a trusted intermediary in the process [25], [26].

3) End-to-End User Identity Authentication: In a scenario where A initiates an access request, if multiple users collectively seek access to B, B must perform individual verification for each message $\mathsf{m_i}$ to ascertain the identity of each user $\mathsf{A_i}$. This sequential verification process is necessary to authenticate the identities of all users involved [27], [33].

D. Risk and Efficiency Analysis in Cross-Chain Scenarios

1) Information Interoperability: Currently, many cross-system authentication schemes are based on a strong assumption that the cryptographic systems of trust domains or the mechanisms of blockchain systems are the same. However, in real-world scenarios, the cryptographic system of each trust domain is likely different, and each blockchain system has its own authentication and consensus mechanisms. Achieving cross-system authentication, information interoperability is the primary and most challenging issue to address.

2) Single Point of Failure: Current cross-chain authentication schemes based on a single notary place very high demands on the reliability and robustness of the notary. However, if the notary experiences hardware failures, cyberattacks, or some other issues, it can result in a single point of failure, ultimately leading to the failure of the authentication mechanism.

3) Handling Multiple Requests Efficiency: When multiple users $\mathsf{A_i}$ simultaneously initiate cross-chain requests, DB needs to complete the entire authentication process for each user's identity verification. This approach results in a linear relationship between communication costs and the number of request messages, consuming a significant amount of notary resources. Additionally, DB's request processing efficiency is low, leading to delays to multiple request messages from users.

E. MU-PRS Algorithm

This section provides a detailed description of the proposed MU-PRS algorithm based on the bilinear operation and security theory assumptions presented in § II.

1) Primary Features: The most significant feature of this algorithm is that the proxy can convert the signatures of multiple users across systems with different cryptographic mechanisms, thereby enabling users in other systems to simultaneously verify these signatures. (d)-(f) is the core algorithm for realizing cross-chain authentication of multiple users in § III-G3. The MU-PRS algorithm has the following phases:

(a) ***Setup***$(\theta \rightarrow pa)$: In the System Initialization stage, the security parameter values are input to generate the system parameters pa.

- Select a large prime number p, elliptic addition $\mathsf{G_1}$ and multiplicative group $\mathsf{G_2}$. Both $\mathsf{G_1}$ and $\mathsf{G_2}$ are of order p, g is a generator of $\mathsf{G_1}$, construct a bilinear mapping e: $\mathsf{G_1} \times \mathsf{G_1} \rightarrow \mathsf{G_2}$, $\mathsf{g_1}, \mathsf{g_2} \in \mathsf{G_1}$ are chosen. Select the collision-resistant hash function H: $\{0,1\}^* \rightarrow \mathsf{Z_p^*}$; $\mathsf{H_2}$:$\{0,1\}^* \rightarrow \{0,1\}^*$.
- Open system parameters $\mathsf{pa} = \{\mathsf{G_1}, \mathsf{G_2}, \mathsf{g}, \mathsf{e}, \mathsf{p}, \mathsf{g_1}, \mathsf{g_2}\}$.

(b) ***KeyGen***$(r_{A_i}, s \rightarrow sk_{A_i}, pk_{A_i})$ / ***KeyGen***$(r_{B_i}, ID_{B_i} \rightarrow sk_{B_i}, pk_{B_i})$: In the Key Generation stage, input parameter values, and output public keys and private keys of $\mathsf{A_i}$ and $\mathsf{B_i}$.

- CLC cryptosystem: The $\mathsf{Blockchain_A}$ randomly selects $\mathsf{x_A} \in \mathsf{Z_p^*}$, Calculating eigenvalue $\mathsf{R} = \mathsf{H}(\mathsf{x_A})$ and disclose it, $\mathsf{KGC_A}$ randomly selects $\mathsf{r_{A_i}} \in \mathsf{Z_p^*}$ and calculates the partial private key $\mathsf{D_{A_i}} = \mathsf{r_{A_i}}$ for user $\mathsf{A_i}$. The secret value s is selected randomly by user $\mathsf{A_i}$, calculate $\mathsf{H(s)} = \mathsf{r_{1i}}$. Allow the private key of user $\mathsf{A_i}$, $\mathsf{sk_{A_i}} = \mathsf{r_{1i}} + \mathsf{D_{A_i}}$, and calculate the public key of user $\mathsf{A_i}$, $\mathsf{pk_{A_i}} = \mathsf{g_1}^{\mathsf{sk_{A_1}}}$.
- IBC cryptosystem: The $\mathsf{KGC_B}$ of $\mathsf{Blockchain_B}$ selects $\mathsf{r_{B_i}} \in \mathsf{Z_p^*}$, make the private key $\mathsf{sk_{B_i}} = \mathsf{r_{Bi}}$ of user $\mathsf{B_i}$. The $\mathsf{KGC_B}$ calculates $\mathsf{H(ID_{B_i})} = \mathsf{r_{2_i}}$ according to identity $\mathsf{ID_{B_i}}$, and the public key $\mathsf{pk_{B_i}} = \mathsf{r_{2_i}}^{\mathsf{sk_{B_i}}}$.

(c) ***Sign***$(AT_{A_i}, sk_{A_i}, m, pk_{A_i}, PK, R \rightarrow \delta_A)$: In the Signature Generation stage, suppose the message is m, the user's attribute set is $\mathsf{AT_{A_i}}$, and the private key is $\mathsf{sk_{A_i}}$. If the $\mathsf{AT_{A_i}}$ of user $\mathsf{A_i}$ satisfies the access policy T, it can sign m with its own private key. Otherwise, it cannot sign m. The steps used to generate a signature are as follows.

- Allow $\mathsf{PK} = \prod_{\mathsf{i}} \mathsf{pk_{A_i}}$.
- Create the interrogation $\mathsf{E} = \mathsf{H}(\mathsf{H_2}\{\mathsf{e}(\mathsf{R}, \mathsf{PK})\}||\mathsf{m})$.
- The signature of user $\mathsf{A_i}$ on message m: $\delta_{\mathsf{A_i}} = \mathsf{R} \times \mathsf{g_2}^{\mathsf{sk_{A_i}}} + \mathsf{E} \times \mathsf{g_2}^{\mathsf{sk_{A_i}}}$.
- Calculate the aggregated signature of i users: $\delta_{\mathsf{A}} = \Sigma_{\mathsf{i}} \delta_{\mathsf{A_i}}$.

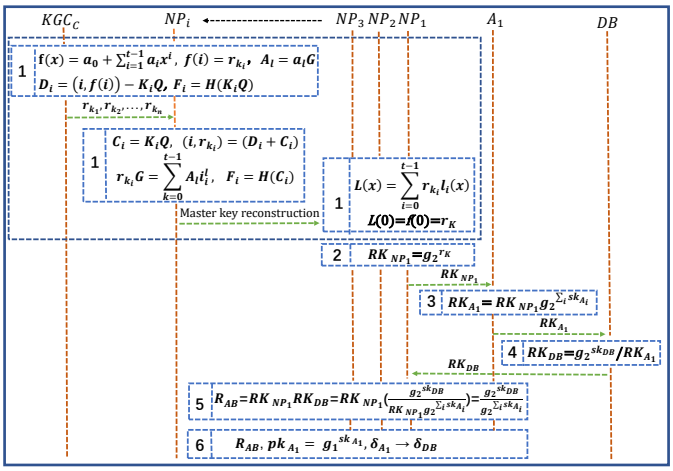


Fig. 3: Flowchart of the signature conversion process in the MN-SC mechanism

(d) ***Re-KeyGen***$(sk_{A_i}, sk_{B_i} \rightarrow R_{AB_i})$: In the Re-signature Key Generation stage, input the private key $\mathsf{sk_{A_i}}$, aggregate the private keys of i users $\mathsf{A_i}$, $\mathsf{i} \in [1, \mathsf{n}]$ and calculate the re-signature key $\mathsf{R_{AB_i}} = \mathsf{g_2^{sk_{Bi}}}/\mathsf{g_2^{\sum_i sk_{Ai}}}$ with the private key $\mathsf{sk_{B_i}}$.

(e) ***Re-Sign***$(\delta_A, R_{AB_i} \rightarrow \delta_{B_i})$: In the Re-signature Generation stage, input the re-signature key $\mathsf{R_{AB_i}}$, the signature of the message δ_A, generate the re-signature:

$$\begin{aligned}\delta_{\mathsf{B_i}} &= \delta_\mathsf{A} \times \mathsf{R_{AB_i}} \\ &= \sum_\mathsf{i} \frac{\delta_{\mathsf{A_i}} \times \mathsf{g_2^{sk_{B_i}}}}{\mathsf{g_2^{\sum_i sk_{A_i}}}} \\ &= \sum_\mathsf{i} \left(\mathsf{R} \times \mathsf{g_2^{sk_{A_i}}} + \mathsf{E} \times \mathsf{g_2^{sk_{A_i}}}\right) \times \frac{\mathsf{g_2^{sk_{B_i}}}}{\mathsf{g_2^{\sum_i sk_{A_i}}}} \\ &= \mathsf{R} \times \mathsf{g_2^{sk_{B_i}}} + \mathsf{E} \times \mathsf{g_2^{sk_{B_i}}}\end{aligned} \tag{6}$$

(f) ***Verify***$(\delta_{B_i}, pk_{B_i}, sk_{B_i} \rightarrow 0/1)$: In the Signature Verification stage, input re-signature $\delta_{\mathsf{B_i}}$, public key $\mathsf{pk_{B_i}}$ of user $\mathsf{B_i}$ and private key $\mathsf{sk_{B_i}}$ to verify whether the equation $\mathsf{e}(\delta_{\mathsf{B_i}}, \mathsf{r_{2i}}) = \mathsf{e}(\mathsf{Rg_2}, \mathsf{pk_{B_i}})\mathsf{e}(\mathsf{Eg_2}, \mathsf{pk_{B_i}})$ holds. If the equation holds, output 1. Otherwise, output 0.

2) Correctness: The signatures that have undergone re-signature key conversion will form re-signatures. Successful verification of the re-signatures by users indicates the correctness of the algorithm. The correctness of the re-signature is as follows:

$$\begin{aligned}\mathsf{e}(\delta_{\mathsf{B_i}}, \mathsf{r_{2i}}) &= \mathsf{e}(\mathsf{Rg_2^{sk_{B_i}}}, \mathsf{r_{2i}})\mathsf{e}(\mathsf{Eg_2^{sk_{B_i}}}, \mathsf{r_{2i}}) \\ &= \mathsf{e}(\mathsf{Rg_2}, \mathsf{r_{2i}^{sk_{B_i}}})\mathsf{e}(\mathsf{Eg_2}, \mathsf{r_{2i}^{sk_{B_i}}}) \\ &= \mathsf{e}(\mathsf{Rg_2}, \mathsf{pk_{B_i}})\mathsf{e}(\mathsf{Eg_2}, \mathsf{pk_{B_i}})\end{aligned} \tag{7}$$

F. MN-SC Mechanism

This paper proposes a Multi-Notary Signature Conversion (MN-SC) mechanism based on the cross-chain communication mechanism from reference [34] and the signature conversion mechanism from reference [26]. The main function of the MN-SC mechanism is to allow inter-chain notaries to obtain access permission to blockchain systems, and then convert the user's signature through the notary. The MN-SC mechanism only requires a certain number of notaries to work normally, effectively solving the single point of failure problem with notaries.

To obtain access permission to the blockchain system, we use the cross-chain communication mechanism from reference [34] as a basis. Differently, when user $\mathsf{A_i}$ sends a cross-chain access request, it also sends an access permission request message Access to its blockchain node. After verifying the user's identity, the node searches for information about the inter-chain notary $\mathsf{NP_i}$ in the system and sends a license $\mathsf{Lic_i}$ to $\mathsf{NP_i}$. Similarly, after receiving the access request and successfully authenticating $\mathsf{A_i}$'s identity, user $\mathsf{B_i}$ obtains blockchain system access permission in the same way. After obtaining access permission to the blockchain, the two blockchains can communicate information. However, due to the different cryptographic systems in each trust domain, it is necessary to convert the signature of a user in trust domain A into a signature recognizable by trust domain B, which needs a signature conversion. The specific steps of the signature conversion process by the MN-SC mechanism are as follows, as shown in Figure 3.

(a) The i cross-chain notaries generate the signature conversion master key $\mathsf{r_K}$ and stores it within notary $\mathsf{NP_1}$.

(b) $\mathsf{NP_1}$ computes $\mathsf{RK_{NP_1}} = \mathsf{H}(\mathsf{r_K})$ and sends it to user $\mathsf{A_1}$.

(c) The $\mathsf{A_1}$ utilizes their aggregated private key SK to calculate $\mathsf{RK_{A_1}} = \mathsf{RK_{NP_1}}\mathsf{g_2^{\sum_i sk_{A_i}}}$ and forwards it to database $\mathsf{B_1}$.

(d) The $\mathsf{B_1}$ calculates $\mathsf{RK_{B_1}} = \mathsf{g_2^{sk_{B_1}}/RK_{A_1}}$ using its own private key and sends it to $\mathsf{NP_1}$.

(e) The $\mathsf{NP_1}$ calculates the re-signature key $\mathsf{R_{AB}} = \mathsf{RK_{NP_1}RK_{B_1}} = \mathsf{RK_{NP_1}}(\frac{\mathsf{g_2^{sk_{B_1}}}}{\mathsf{RK_{NP_1}g_2^{\sum_i sk_{A_i}}}}) = \frac{\mathsf{g_2^{sk_{B_1}}}}{\mathsf{g_2^{\sum_i sk_{A_i}}}}$ based on $\mathsf{RK_{NP_1}}$.

(f) Upon receiving user $\mathsf{A_1'}$s signature $\delta_{\mathsf{A_1}}$, $\mathsf{NP_1}$ employs the re-signature key $\mathsf{R_{AB}}$ to calculate the re-signature $\delta_{\mathsf{B_1}}$, thus completing the signature conversion.

In the MN-SC mechanism, steps (a)-(e) can be completed during the system initialization phase, and the generated re-signature keys $\mathsf{R_{AB}}$ are stored in $\mathsf{NP_1}$. Each cross-chain notary generates a signature conversion master key $\mathsf{r_K}$. The generation process for $\mathsf{r_K}$ in MN-SC mechanism is as follows:

(a) Initialization stage
Distributor $\mathsf{KGC_C}, \mathsf{NP_i} = \{\mathsf{NP_1}, \mathsf{NP_2}, \ldots, \mathsf{NP_n}\}$, master key is $\mathsf{r_k}$. The distributor $\mathsf{KGC_C}$ randomly selects n points $\mathsf{K_1}, \mathsf{K_2}, \ldots, \mathsf{K_n}$, on the elliptic curve $\mathsf{E_p(a, b)}$, and makes its vertical coordinate $\mathsf{r_{k_1}}, \mathsf{r_{k_2}}, \ldots, \mathsf{r_{k_n}}$ as the subkey of $\mathsf{NP_i}$, G is the base point selected on $\mathsf{E_p(a, b)}$. $\mathsf{KGC_C}$ distributes its subkeys to $\mathsf{NP_i}, \mathsf{i} \in [1, \mathsf{n}]$ over a secure channel. $\mathsf{KGC_C}$ calculates $\mathsf{G_i'} = \mathsf{k_i G}, \mathsf{i} \in [1, \mathsf{n}]$. Then, parameters $(\mathsf{E, G, n, H(x), G'})$ is disclosed.

(b) Key distribution

- Distributor $\mathsf{KGC_C}$ randomly selects a point Q on $\mathsf{E_p(a, b)}$ and a $\mathsf{t-1}$ degree polynomial $\mathsf{f(x)} = \mathsf{a_0} + \sum_{\mathsf{i}=1}^{\mathsf{t}-1} \mathsf{a_i x^i}$, where $\mathsf{f(0)} = \mathsf{a_0} = \mathsf{r_K}$ and Q is the public parametlates and $\mathsf{f(x)}$ is retained by the distributor.

- Distributor KGC_C calculates $f(i) = r_{k_i}$, $A_l = a_l G$, $1 \le l \le t-1$, $D_i = (i, f(i)) - K_i Q$. The parameters A_l and D_i are disclosed.
- Distributor KGC_C calculates $F_i = H(K_i Q)$, which is used to verify the authenticity of the subkeys interacted by the NP_i.

(c) Verification of subkeys
After receiving the subkey distributed by KGC_C, $NP_i (1 \le i \le n)$ first verifies the authenticity of the subkey. Then it calculates $C_i = K_i Q$ and $(i, r_{k_i}) = (D_i + C_i)$. If the equation $r_{k_i} G = \sum_{k=0}^{t-1} A_l i_i^l$ holds, then the subkey is true. If the equation $F_i = H(C_i)$ holds, it means that the identity between NP_i is true.

(d) Master key reconstruction
When the number l of participants satisfies $l \ge t$, allow the participants be $NP_{i_1}, NP_{i_2}, \ldots NP_{i_l}$, and the master key r_K can be reconstructed by Lagrange interpolation. For the subkeys $r_{k_i}, i \in [1, t]$ of t different NP_i, a unique Lagrange interpolation polynomial $L(x) = \sum_{i=0}^{t-1} r_{k_i} l_i(x)$ can be constructed by (x_i, r_{k_i}), where $l_i(x) = \prod_{0 \le j \le t-1} \frac{x - x_i}{x_i - x_j}, 0 \le i \le t-1$. Because $L(x)$ is a polynomial of degree $t-1$ and corresponds to different values, there is $L(0)=f(0)=r_k$. Get the master key and send r_k to NP_1.

G. MU-CCAS

In this section, MU-CCAS is constructed based on MU-PRS and MN-SC, and the MU-CCAS model is described. The MU-CCAS model consists of five entities, as shown in Figure 2.

1) System Initialization: Before initiating a cross-chain access request, the system will undergo an initialization process.

(1.1) *System Parameter Generation*: $Blockchain_i, KGC_i$ employ a System Initialization algorithm (***Setup***) of MU-PRS in § III-E based on the elliptic curve $Ep(a, b)$ to generate the system parameters. These system parameters are then publicly disclosed as $pa = \{G_1, G_2, g, e, p, g_1, g_2, H, H_1, H_2\}$.

(1.2) *Identity Generation*: The layered identity tree is effectively employed to ensure identity uniqueness. This solution adopts the layered identity tree structure outlined in reference [32] to establish the identities of A_i and DB. Consequently, the actual identity of A_i can be represented as $ID_{A_i} = DN_\alpha \| DN_{A_i}$. The actual identity of DB is given by $ID_{DB} = DN_\beta || DN_{DB}$. The actual identity of NP_1 is represented as $ID_{NP_i} = DN_\theta || DN_{NP_i}$.

(1.3) *Key Generation*: KGC uses Key Generation algorithm (***KeyGen***) of MU-PRS to generate keys for entities in the domain.

- *Key Generation at KGC*: For KGC_A in Hospital A: $\alpha \in Z_p^*$ as the main secret key $SK_A = \alpha$ for KGC_A. Then the public key $PK_A = {g_1}^\alpha$ for KGC_A is computed. Similarly, for KGC_B in Hospital B: Generate its main secret key $SK_B = \beta$ and public key $PK_B = g_1^\beta$. Also, generate the main secret key $SK_C = \theta$ and public key $PK_C = {g_1}^\theta$ for NP_1. Finally, publicly disclose the public keys PK_A, PK_B, and PK_C.
- *Key Generation of* A_i *and* DB: A_i in Hospital A randomly selects a secret value $t_{A_i} \in Z_p^*$, and computes the temporary identity $TID_{A_i} = H_1(ID_{A_i} || {g^{t_{A_i}}})$. The A_i sends the request message to apply for keys to KGC_A. KGC_A verifies the correctness of TID_{A_i}. Then, KGC_A randomly selects $r_{A_i}, x_A \in Z_p^*$, calculates the partial private key $D_{A_i} = r_{A_i}$. $R = H(x_A)$ serves as a feature identifier within $Blockchain_A$. Subsequently, KGC_A computes the signature $\delta_{A \to A_i} = (H_1(TID_{A_i}))^a$ for TID_{A_i}. Finally, the local timestamp $T^0_{A_1}$ is read, and the message $\{D_{A_i}, R, x_A, T^0_{A_i}, \delta_{A \to A_i}\}$ is returned to A_i via a secret channel. After receiving the returned message, A_i randomly chooses the secret value $r_{1i} \in Z_p^*$, calculates its complete private key $sk_{A_i} = r_{1i} + D_{A_i}$ and public key $pk_{A_i} = g_1^{sk_{A_1}}$. Finally, pk_{A_i} is disclosed. Similarly, KGC_B randomly select $r_{B_i} \in Z_p^*$, and based on the identity ID_{DB}, calculate $H(TID_{DB}) = r_{2_i}$, where the user DB's private key is $sk_{DB} = r_{DB}$, and the public key is $pk_{DB} = r_{2_i}^{sk_{DB}}$.
- *Generation of Aggregate Private Key and Aggregate Public Key*: In Hospital A, each users $A_i, i \in [1, n]$ utilizes his respective private keys sk_{A_i} to calculate individual values $sk_i = {g_2}^{sk_{A_i}}$, and send sk_i to user A_1. A_1 calculates the aggregated secret key $SK = g_2 \Sigma_i sk_{A_i}$ and the aggregated public key $PK = \prod_i pk_{A_i}$.

2) Multi-User Cross-Chain Access: After the system initialization is completed, assuming that i users in Hospital A have collected their aggregated private keys and public keys in A_1, these i users in Hospital A need to send cross-chain access requests to the database DB in Hospital B. In this paper, the Signature Generational algorithm (***Sign***) of MU-PRS is used to generate the signature of each user for the message m. These signatures from i users are then aggregated into an aggregated signature δ_A, which is sent to DB. This aggregation method eliminates the need for the DB to repeatedly verify the signatures of i users, greatly improving the efficiency of verification. The smart contract algorithm is shown in Algorithm 1.

(2.1) *Sending Cross-Chain Access Requests*: User A_1 randomly picks $y \in Z_p^*$ and calculates the key negotiation parameter $Y' = g^y$. Then, the TID_{A_1} is input with the password pw to calculate w and m, where $T^1_{A_1}$ is the timestamp, $N^1_{A_1}$ is a random parameter to keep the message fresh. δ_{A_1} is the signature of A_1 to message m, $\delta_{A_1} = R \times {g_2}^{sk_{A_1}} + E \times {g_2}^{sk_{A_i}}$ where $E = H(H_2\{e(R, PK)\} || m)$. Later the aggregated signatures of i users are calculated within the blockchain BC_A for message m: $\delta_A = \sum_i \delta_{A_i}, i \in [1, n]$. Finally A_1 sends the message m_{A_1} to DB.

(2.2) *Verifying the Request Message*: Upon receiving the message, the DB judges whether $request_1$ is an access request. If not, reject it; or query user information in the user access list according to TID_{A_1}. If the query to the user information is in the list, then directly repeat the cross-chain authentication operation. Then, the DB check the freshness of timestamp T_{A_1} and verify that

Algorithm 1: Multi-User Cross-Chain Access

Input: public parameters, $\mathsf{TID_{A_1}}$, pw, y, e, y, $\mathsf{T^1_{A_1}}$, $\mathsf{N^1_{A_1}}$
Output: $\mathsf{Y'} = \mathsf{g^y}$, $\mathsf{w} = \mathsf{H_2(TID_{A_1}||pw)}$, True/False

begin
  /* $\mathsf{A_1}$ **prepares:** */
  $\mathsf{w} \leftarrow \mathsf{H_2(TID_{A_1}||pw)}$
  $\mathsf{e} = \mathsf{H(H_2\{e(R, PK)\}||m)}$
  $\delta_{\mathsf{A_1}} \leftarrow \mathsf{Sign_{A_1}}(\mathsf{x_a}, \mathsf{g_2}, \mathsf{sk_{A_1}e})$
  $\delta_{\mathsf{A}} = \sum_{\mathsf{i}} \delta_{\mathsf{A_1}}$ // for $\mathsf{A_i} \in$ user group A
  $\mathsf{m_{A_1}} \leftarrow \mathsf{Encrypt\{request_1, TID_{A_1}, TID_{DB}, T^1_{A_1}, N^1_{A_1}, pk_{A_1}},$
  $\delta_{\mathsf{A}\rightarrow\mathsf{A_1}}, \mathsf{e}, \mathsf{Y'}, \mathsf{m}, \delta_{\mathsf{A}}\}$
  /* $\mathsf{A_1}$ sends $\mathsf{m_{A_1}}$ to DB */
  /* DB **judges:** */
  $\mathsf{JudgeResult} \leftarrow \mathsf{Judge_{DB}(request_1)}$
  /* DB receives $\mathsf{request_1}$ and decrypts $\mathsf{m_{A_1}}$ */
  $\mathsf{FindFlag} \leftarrow \mathsf{Find_{DB}(TID_{A_1})}$
  $\mathsf{IsFresh_{DB}} \leftarrow \mathsf{CheckFreshness_{DB}(T^1_{A_1})}$
  if !JudgeResult *or* !FindFlag *or* $\mathsf{!IsFresh_{DB}}$ **then**
    return *False*
  end
  if $\mathsf{e}(\delta_{\mathsf{A}\rightarrow\mathsf{A_1}}, \mathsf{g_1}) == \mathsf{e(H_1(TID_{A_1}), PK_A)}$ **then**
    return *True* // Goto *Algorithm2*
  end
  else
    return *False*
  end
end

Algorithm 2: Multi-Notary Signature Conversion

Input: public parameters, $\mathsf{pk_{DB}}$, $\mathsf{sk_{DB}}$, $\mathsf{pk_{A_1}}$, $\mathsf{R_{AB}}$, δ_{A}, $\mathsf{pk_{NP_1}}$, $\mathsf{N^1_{DB}}$, $\mathsf{T^1_{DB}}$, δ_{DB}
Output: h, True/False

begin
  /* DB **prepares:** */
  $\mathsf{m_{DB}} \leftarrow \mathsf{Encrypt\{TID_{DB}, T^1_{DB}, N^1_{DB}, pk_{A_1}, pk_{DB}}, \delta_{\mathsf{A}}\}$
  /* DB sends $\mathsf{m_{DB}}$ to Notary Public $\mathsf{NP_1}$ */
  /* $\mathsf{NP_1}$ **does:** $\mathsf{NP_1}$ first receives $\mathsf{m_{DB}}$ and then does as follows */
  $\mathsf{IsFresh_{NP_1}} \leftarrow \mathsf{CheckFreshness_{NP_1}(N^1_{DB}, T^1_{DB})}$
  $\mathsf{LegalFlag} \leftarrow \mathsf{IsLegal(identity_{DB})}$
  /* Check whether the identity of DB is legal */
  if $\mathsf{!IsFresh_{NP_1}}$ *or* !LegalFlag **then**
    return *False*
  end
  $\delta_{\mathsf{DB}} \leftarrow \mathsf{Mul}(\delta_{\mathsf{A}}\mathsf{R_{AB}})$
  $\mathsf{h} \leftarrow \mathsf{H_1(TID_{NP_1})}$
  $\mathsf{m_{NP_1}} \leftarrow \mathsf{Encrypt\{TID_{NP_1}, TID_{DB}, T^1_{NP_1}, N^1_{DB}, h, pk_{NP_1}}, \delta_{\mathsf{DB}}\}$
  /* $\mathsf{NP_1}$ sends response information $\mathsf{m_{NP_1}}$ to DB */
  /* DB **responses:** DB first receives $\mathsf{m_{NP_1}}$ and decrypts it */
  $\mathsf{IsFresh_{DB}} \leftarrow \mathsf{CheckFreshness_{DB}(T^1_{NP_1})}$
  if $\mathsf{!IsFresh_{DB}}$ **then**
    return *False*
  end
  if $\mathsf{e}(\delta_{\mathsf{DB}}, \mathsf{g_1}) == \mathsf{e(R, pk_{DB})e(g_2, pk_{DB})}$ **then**
    Save $\{\mathsf{TID_{A_1}}, \mathsf{w}, \mathsf{pk_{A_1}}, \mathsf{N_1}, \mathsf{D_1}\}$ in user access list
    return *True* // Goto *Algorithm3*
  end
  else
    return *False*
  end
end

$\mathsf{e}(\delta_{\mathsf{A}\rightarrow\mathsf{A_1}}, \mathsf{g}) = \mathsf{e(H_1(TID_{A_1}), PK_A)}$ is valid.

To obtain blockchain system access permission, when user $\mathsf{A_1}$ sends an access request to DB, they also send an access permission request message $\{\mathsf{Access_1}, \mathsf{TID_{A_1}}, \mathsf{pk_{A_1}}, \delta_1\}$ to Hospital A. After verifying user $\mathsf{A_1}$'s identity, Hospital A searches for information on the inter-chain notary $\mathsf{NP_1}$ in the system, sends a license $\mathsf{Lic_1} = \{\mathsf{Licence_1}, \mathsf{TID_{A_1}}, \mathsf{pk_{A_1}}, \mathsf{PK_A}, \delta_2\}$ to $\mathsf{NP_1}$, and stores the information $\{\mathsf{TID_{A_1}}, \mathsf{pk_{A_1}}, \mathsf{PK_C}, \mathsf{N_A}, \mathsf{D_A}\}$ in the blockchain, where $\mathsf{N_A}$ and $\mathsf{D_A}$ represent the number of accesses by $\mathsf{NP_1}$ and the validity period, respectively. Similarly, after receiving the access request and successfully verifying the identity of $\mathsf{A_1}$, DB sends the access permission request message to Hospital B. After a series of verifications, the Hospital B sends a license $\mathsf{Lic_2}$ to $\mathsf{NP_1}$ and stores the information $\{\mathsf{TID_{DB}}, \mathsf{pk_{DB}}, \mathsf{PK_C}, \mathsf{N_B}, \mathsf{D_B}\}$ in the blockchain. This approach ensures that interactions between $\mathsf{A_1}$ or DB and $\mathsf{NP_1}$ are forwarded and duly documented by their respective affiliated nodes, obviating the need for in-depth explanation within this context.

3) Multi-Notary Signature Conversion: The DB sends a signature conversion request to $\mathsf{NP_1}$, which uses the Re-signature Key Generation algorithm (***Re-KeyGen***) of MU-PRS and the MN-SC mechanism to perform the Re-signature Generation algorithm (***Re-Sign***) of MU-PRS, converting the aggregated signature δ_{A} from Hospital A into a signature δ_{DB} that the DB can verify. The corresponding smart contract algorithm is shown in Algorithm 2.

(3.1) *Sending Signature Conversion Request*: The DB proceeds by randomly selecting $\mathsf{N_{DB}}^1 \in \mathsf{Z_p^*}$ and retrieving the timestamp $\mathsf{T_{DB}}^1$. It then dispatches the application conversion message $\mathsf{m_{DB}}$ to $\mathsf{NP_1}$. It's important to note that DB initially transmits the message to its node within Hospital B, which subsequently relays it to $\mathsf{NP_1}$. Concurrently, Hospital B records this operation in the $\mathsf{Blockchain_B}$.

(3.2) *Generating the Re-Signature*: Upon receiving the message, $\mathsf{NP_1}$ checks the freshness of the random parameter $\mathsf{N^1_{DB}}$ and the timestamp $\mathsf{T^1_{DB}}$. Then, the $\mathsf{NP_1}$ uses the Re-signature Generation algorithm (***Re-Sign***) of MU-PRS to calculate the re-signature:

$$\begin{aligned}\delta_{\mathsf{DB}} &= \delta_{\mathsf{A}}\mathsf{R_{AB}} \\ &= \sum_{\mathsf{i}} \delta_{\mathsf{A_i}} \times \mathsf{g_2}^{\mathsf{sk_{DB}}} / \mathsf{g_2}^{\Sigma_{\mathsf{i}}\mathsf{sk_{A_i}}} \\ &= \sum_{\mathsf{i}} \left(\mathsf{R} \times \mathsf{g_2}^{\mathsf{sk_{A_i}}} + \mathsf{E} \times \mathsf{g_2}^{\mathsf{sk_{A_i}}}\right) \times \mathsf{g_2}^{\mathsf{sk_{DB}}} / \mathsf{g_2}^{\sum_{\mathsf{i}} \mathsf{sk_{A_i}}}. \\ &= \mathsf{R} \times \mathsf{g_2}^{\mathsf{sk_{DB}}} + \mathsf{E} \times \mathsf{g_2}^{\mathsf{sk_{DB}}}\end{aligned}$$

The $\mathsf{NP_1}$ cannot calculate the private keys or partial private keys of $\mathsf{A_1}$ and the DB using the re-signature key. It can only perform signature conversion on the cross-chain request messages. Finally, the $\mathsf{NP_1}$ calculates $\mathsf{H_1(TID_{NP_1})}$, reads the timestamp $\mathsf{T^1_{NP_1}}$, and sends the response message $\mathsf{m_{NP_1}}$ to the DB. It should be noted that the re-signature key $\mathsf{R_{AB}}$ is generated by the Re-signature Key Generation algorithm (***Re-KeyGen***) of MU-PRS, following the steps outlined in § III-F for the MN-SC mechanism (steps (a)-(e)). This process can be completed before initiating the access. After the re-signature key is generated, it is stored in $\mathsf{NP_1}$, while $\mathsf{NP_1}$ cannot identify the private keys of $\mathsf{A_i}$ and the DB.

(3.3) *Verifying the Re-Signature*: The DB authentication of the $\mathsf{N^1_{DB}}$, $\mathsf{T^1_{NP_1}}$ and uses the Signature Verification

Algorithm 3: Establishing a Session Channel

Input: public parameters, pk_{DB}, sk_{DB}, z, $Z' = g^z$, T^2_{DB}, N^2_{DB}, h
Output: True/False

begin
  /* DB **signs:** */
  $\delta'_{DB} \leftarrow Sign_{DB}(g_2, sk_{DB}, e')$
  $m'_{DB} \leftarrow Encrypt\{request_2, TID_{NP_1}, TID_{DB}, Z', T^2_{DB}, N^2_{DB}, N^1_{A_1}, H_1(TID_{NP_1}), pk_{DB}, pk_{NP_1}, \delta_{B\to DB}, m_1, \delta_{DB}\}_{pk_A}$
  /* DB sends m_{DB} to A_1 */
  /* A_1 **judges:** */
  $Flag \leftarrow (N^1_{A_1}, N^2_{DB}, T^2_{DB})$
  $S \leftarrow IsSame(h, h_{response})$
  if !Flag *or* !S **then**
    return *False*
  end
  /* A_1 saves $\{TID_{NP_1}, TID_{DB}, pk_{NP_1}, pk_{DB}\}$ in user access list */
  $K \leftarrow CalWithExp(pk_{DB}, sk_{A_1}, Z', y)$
  /* A_1 establishes the session channel with the database DB */
  return *True* // End of process
end

algorithm (***Verify***) of MU-PRS to verify the correctness of the signature δ_{DB}. It inputs δ_{DB}, pk_{DB} and sk_{DB} to calculate and verify whether the equation $e(\delta_{DB}, r_{2i}) = e(Rg_2, pk_{DB})e(Eg_2, pk_{DB})$ holds. If the equation holds, outputs 1. Otherwise, it outputs 0. If the verification is successful, δ_{DB} is the legitimate signature of message m. It completes the authentication of A_1 and saves $\{TID_{A_1}, w, pk_{A_1}, N1, D1\}$ in the user access list. N1 and D1 are the number of times and valid time of repeated cross-chain authentication of A_1. Meanwhile, the procedure goes to the next step.

4) Establishing a Session Channel: At this stage, DB calculates the key negotiation parameter $Z' = g^Z$, sends the response message m'_{DB} to A_1, and computes the session key $K = (pk_{A_1})^{sk_{DB}}(Y')^z$. The A_1 upon receiving the response message, authenticates the random parameters N^2_{DB}, timestamp T^2_{DB}, and $H_1(TID_{NP_1})$. Upon successful authentication, the A_1 stores $\{TID_{NP_1}, TID_{DB}, pk_{NP_1}, PK_{DB}\}$ in the user access list and calculates the session key $K = (pk_{DB})^{sk_{A_1}}(Z')^y$. Through these steps, a session channel is established with the DB. The corresponding smart contract algorithm is shown in Algorithm 3.

IV. Security Analysis

In this section, inspired by the security evaluation methodology outlined in [35],[36],[37],[38], we rigorously assess the security of the MU-CCAS from three critical dimensions. Firstly, in § IV-A, we establish the logical correctness of the proposed solution by applying BAN logic. Secondly, in § IV-B, we conduct a comprehensive security evaluation of the MU-CCAS using the formal verification tool *Scyther*. This tool is employed to analyze and verify the scheme's confidentiality, authentication, and integrity. Lastly, in § IV-C, we delve into a security attribute analysis of the MU-CCAS. The collective findings from these security evaluations decisively affirm that the MU-CCAS effectively aligns with security prerequisites and successfully fulfills its security goals.

A. Logical Correctness Proof by BAN logic

BAN logic is a belief-based modal logic that comprises 19 rules.[1] When using BAN logic, the first step is the "modeling phase", where protocol messages are transformed into formulas within the framework. Following this, reasonable assumptions are made based on the specific context. Using the logic's inference rules, one can then reason about whether the protocol can achieve its intended objectives in accordance with the idealized protocol and the assumptions made.

1) Establish an Idealized Protocol Model: Following the protocol's description, a concrete idealized protocol model can be created, consisting of the following four messages:

Message M1:

$A1 \to DB: \{TID_{A_1}, TID_{DB}, T^1_{A_1}, N^1_{A_1}, pk_{A_1}, \delta_{A\to A_1}, m, Y', \delta_A\}_{pk_{DB}}$

Message M2:

$DB \to NP_1: \{TID_{DB}, {T_{DB}}^1, {N_{DB}}^1, pk_{A_1}, pk_{DB}, \delta_A\}_{pk_{NP_1}}$

Message M3:

$NP_1 \to DB: \{TID_{NP_1}, TID_{DB}, {T_{NP_1}}^1, {N_{DB}}^1, H_1(TID_{NP_1}), pk_{NP_1}, \delta_{DB}\}_{pk_{DB}}$

Message M4:

$DB \to A1: \{TID_{NP_1}, TID_{DB}, Z', {T_{DB}}^2, {N_{DB}}^2, N^1_{A_1}, H_1(TID_{NP_1}), pk_{DB}, pk_{NP_1}, \delta_{B\to DB}, m_1, \delta_{DB}\}_{pk_{A_1}}$

2) Setting Goals: According to BAN logic, the goals of the protocol are described in a language that can resist malicious attacks only when the protocol achieves the predetermined objectives. For this protocol, the following four goals are set:

Goal G1: $A1| \equiv DB| \equiv (A1 \overset{K}{\longleftrightarrow} DB)$
Goal G2: $A1| \equiv (A1 \overset{K}{\longleftrightarrow} DB)$
Goal G3: $DB| \equiv A1| \equiv (A1 \overset{K}{\longleftrightarrow} DB)$
Goal G4: $DB| \equiv (A1 \overset{K}{\longleftrightarrow} DB)$

3) Formulating Assumptions: The formulation of assumptions is crucial for the correctness proof of the protocol logic. Thus, following assumptions are made for the model paper:

Assumption a1: $A1| \equiv \#(T^1_{A_1}), A1| \equiv \#({T_{DB}}^2)$
Assumption a2: $DB| \equiv \#(T^1_{A_1}), DB| \equiv \#({T_{DB}}^1), DB| \equiv \#({T_{NP_1}}^1), DB| \equiv \#({T_{DB}}^2)$
Assumption a3: $NP1| \equiv \#({T_{DB}}^1), NP1| \equiv \#({T_{NP_1}}^1)$
Assumption a4: $A1| \equiv A1 \overset{pk_{DB}}{\longleftrightarrow} DB$
Assumption a5: $DB| \equiv DB \overset{pk_{A1}}{\longleftrightarrow} A1, DB| \equiv DB \overset{pk_{NP1}}{\longleftrightarrow} NP1$
Assumption a6: $NP1| \equiv NP1 \overset{pk_{DB}}{\longleftrightarrow} DB$
Assumption a7: $A1| \equiv DB| \Rightarrow A1 \overset{K}{\longleftrightarrow} DB$
Assumption a8: $DB| \equiv A1| \Rightarrow A1 \overset{K}{\longleftrightarrow} DB$

4) Protocol Analysis: Based on the proposed protocol assumptions and the rules of BAN logic, the correctness of the protocol can be proven. The process is as follows:

According to message M1, it can be inferred that:

$$DB \triangleleft \{TID_{A_1}, TID_{DB}, T^1_{A_1}, N^1_{A_1}, pk_{A_1}, \delta_{A\to A_1}, m, Y', \delta_A\}_{pk_{DB}} \quad (8)$$

Based on Assumption a5, it can be concluded that:

$$DB| \equiv A1| \sim (TID_{A_1}, TID_{DB}, T^1_{A_1}, N^1_{A_1}, pk_{A_1}, \delta_{A\to A_1}, m, Y', \delta_A) \quad (9)$$

Based on Assumption a2, it can be concluded that:

$$DB| \equiv \#(TID_{A_1}, TID_{DB}, T^1_{A_1}, N^1_{A_1}, pk_{A_1}, \delta_{A\to A_1}, m, Y', \delta_A) \quad (10)$$

Based on Formula (9), (10), it can be concluded that:

$$DB| \equiv A1| \equiv (TID_{A_1}, TID_{DB}, T^1_{A_1}, N^1_{A_1}, pk_{A_1}, \delta_{A\to A_1}, m, Y', \delta_A) \quad (11)$$

[1]For specific details on inference rules, please refer to [30].

Based on Formula (11), it can be concluded that:

$$\mathsf{DB}| \equiv \mathsf{A1}| \equiv \mathsf{Y}' \tag{12}$$

According to message M4, it can be inferred that:

$$\mathsf{A1} \lhd \{\mathsf{TID}_{\mathsf{NP}_1}, \mathsf{TID}_{\mathsf{DB}}, \mathsf{Z}', {\mathsf{T}_{\mathsf{DB}}}^2, {\mathsf{N}_{\mathsf{DB}}}^2, \mathsf{N}^1_{\mathsf{A}_1}, \mathsf{H}_1(\mathsf{TID}_{\mathsf{NP}_1}), \mathsf{pk}_{\mathsf{DB}}, \mathsf{pk}_{\mathsf{NP}_1}, \delta_{\mathsf{B}\to\mathsf{DB}}, \mathsf{m}_1, \delta_{\mathsf{DB}}\}_{\mathsf{pk}_{\mathsf{A}_1}} \tag{13}$$

Based on Assumption a4, it can be concluded that:

$$\mathsf{A1}| \equiv \mathsf{DB}| \sim (\mathsf{TID}_{\mathsf{NP}_1}, \mathsf{TID}_{\mathsf{DB}}, \mathsf{Z}', {\mathsf{T}_{\mathsf{DB}}}^2, {\mathsf{N}_{\mathsf{DB}}}^2, \mathsf{N}^1_{\mathsf{A}_1}, \mathsf{H}_1(\mathsf{TID}_{\mathsf{NP}_1}), \mathsf{pk}_{\mathsf{DB}}, \mathsf{pk}_{\mathsf{NP}_1}, \delta_{\mathsf{B}\to\mathsf{DB}}, \mathsf{m}_1, \delta_{\mathsf{DB}}) \tag{14}$$

Based on Assumption a1, it can be concluded that:

$$\mathsf{A1}| \equiv \#(\mathsf{TID}_{\mathsf{NP}_1}, \mathsf{TID}_{\mathsf{DB}}, \mathsf{Z}', {\mathsf{T}_{\mathsf{DB}}}^2, {\mathsf{N}_{\mathsf{DB}}}^2, \mathsf{N}^1_{\mathsf{A}_1}, \mathsf{H}_1(\mathsf{TID}_{\mathsf{NP}_1}), \mathsf{pk}_{\mathsf{DB}}, \mathsf{pk}_{\mathsf{NP}_1}, \delta_{\mathsf{B}\to\mathsf{DB}}, \mathsf{m}_1, \delta_{\mathsf{DB}}) \tag{15}$$

Based on Formula (14) and (15), it can be concluded that:

$$\mathsf{A1}| \equiv \mathsf{DB}| \equiv (\mathsf{TID}_{\mathsf{NP}_1}, \mathsf{TID}_{\mathsf{DB}}, \mathsf{Z}', {\mathsf{T}_{\mathsf{DB}}}^2, {\mathsf{N}_{\mathsf{DB}}}^2, \mathsf{N}^1_{\mathsf{A}_1}, \mathsf{H}_1(\mathsf{TID}_{\mathsf{NP}_1}), \mathsf{pk}_{\mathsf{DB}}, \mathsf{pk}_{\mathsf{NP}_1}, \delta_{\mathsf{B}\to\mathsf{DB}}, \mathsf{m}_1, \delta_{\mathsf{DB}}) \tag{16}$$

Based on Formula (16), it can be concluded that:

$$\mathsf{A1}| \equiv \mathsf{DB}| \equiv \mathsf{Z}' \tag{17}$$

DB can calculate $\mathsf{K} = (\mathsf{pk}_{\mathsf{A}_1})^{\mathsf{sk}_{\mathsf{DB}}}(\mathsf{Y}')^{\mathsf{Z}}$, A1 can calculate $(\mathsf{pk}_{\mathsf{DB}})^{\mathsf{sk}_{\mathsf{A}_1}}(\mathsf{Z}')^{\mathsf{y}}$, according to Formula (12) and Formula (17), we can get Formula (18) and Formula (19) by using the obtained values:

$$\mathsf{DB}| \equiv \mathsf{A1}| \equiv (\mathsf{A1} \overset{\mathsf{K}}{\longleftrightarrow} \mathsf{DB}) \tag{18}$$

Formula (18) satisfies Goal G3.

$$\mathsf{A1}| \equiv \mathsf{DB}| \equiv (\mathsf{A1} \overset{\mathsf{K}}{\longleftrightarrow} \mathsf{DB}) \tag{19}$$

Formula (19) satisfies Goal G1.

Based on Assumption a8 and Formula (18), it can be concluded that:

$$\mathsf{DB}| \equiv (\mathsf{A1} \overset{\mathsf{K}}{\longleftrightarrow} \mathsf{DB}) \tag{20}$$

Formula (20) satisfies Goal G4.

Based on Assumption a7 and Formula (14), it can be concluded that:

$$\mathsf{A1}| \equiv (\mathsf{A1} \overset{\mathsf{K}}{\longleftrightarrow} \mathsf{DB}) \tag{21}$$

Formula (21) satisfies Goal G2.

The above modeling and analysis process omits the temporary identities TID_i and public keys pk_i of both parties.

In conclusion, Formula (18), (19), (20) and (21) satisfy the four expected goals set by the protocol model, which proves the logical correctness, confidentiality, integrity of the protocol and thus its rationality.

B. Formal Verification by Scyther Tool

In the preceding section, we have the logical correctness of the proposed scheme by using BAN logic. In this section, we enhance the reliability of BAN logic reasoning by employing the *Scyther* [39]. The *Scyther* tool leverages the heuristic algorithm Athena [40] to perform automated verification of security protocol attacks, thereby substantiating the protocol's correctness. To formally model the MU-CCAS, on the basis of modeling, long-term key leakage, session key leakage, random number leakage and other attack types are added to verify the security attributes such as confidentiality, integrity and authentication property of the MU-CCAS. [2]

[2]The Scyther code associated with this paper has been made available on GitHub at the following URL

1) Scyther tool: Scyther is a formal verification tool designed to analyze and validate the security properties of protocols, encompassing attributes such as confidentiality, authenticity, and integrity. For authentication purposes, *Scyther* integrates with the Security Protocol Description Language (SPDL), enabling specification of protocol designs and verification of intended security assumptions. *Scyther* effectively scans and identifies potential security vulnerabilities within the protocol based on the defined assumptions.

2) Formal Protocol Modeling: To model the proposed scheme, we have established three distinct roles: A_1, DB and NP_1. A_1 represents a user within $\mathsf{Blockchain}_\mathsf{A}$, DB signifies a database within $\mathsf{Blockchain}_\mathsf{B}$, and NP_1 denotes a notary between the interlinked chains.

(a) *Entity Declarations*: Where "*hashfunction*" represents the built-in hash function, and "*usertype*" signifies a user-defined type. The entity declarations are *usertype text*, *usertype Timestamp* and *hashfunction H*.

(b) *Modeling and Implementation*: The modeling and implementation process is shown in Table III. Steps 1-3 depict A_1 initiating a cross-chain access request to DB. Steps 4-8 signify the completion of signature conversion operations between DB and NP_1. Steps 9-11 establish a session channel between DB and A_1. Therefore, we have achieved formal method modeling of the protocol based on this.

3) Formal Security Property Modeling: Scyther as a formal verification tool, doesn't directly provide formal verification for authenticity. It necessitates verifying its authenticity through Secret, Aliveness (*Alive*), Weak Agreement (*Weakagree*), Non-injective Agreement (*Niagree*), and Non-injective Synchronization (*Nisynch*). The strength of authenticity increases sequentially for *Alive*, *Weakagree*, *Niagree* and *Nisynch*. *Alive* represents liveness authentication, serving as a fundamental form of authentication that ensures the expected communicating party A exists. *Weakagree*, denoting weak protocol authentication, demands that certain states or values among participating parties remain consistent throughout the protocol's execution. *Niagree*, referring to non-monotonic agreement authentication, describes how communication or negotiation outcomes between parties cannot be repudiated during protocol execution. *Nisynch* authentication, indicates that, in the event an attacker gains access to the private key of proxy A, all send/receive events preceding a claimed event are correctly executed by proxy A in the correct order and content. This property ensures the integrity of received messages, *Nisynch*'s definition is quite similar to *Niargree*'s, with the distinction that *Nisynch* adds a requirement for expected order, thus exhibiting stronger authenticity. The key source code of formal verification of security attributes is shown in Table IV.

Four attributes relevant to this paper, there are the following first-order logic expressions. We take entity A_1 as an example to demonstrate them:

(a) *Alive:*

$$\mathsf{claim}(\mathsf{A}_1, \mathsf{forall\ x : Alive(x)})$$

In this expression, the keyword "claim" is used to declare a property statement. "forall x" signifies "for all entities

TABLE III: The modeling and implementation process in scyther

Modeling steps	Modeling pseudocode

1.A_1 performs computational operations.	match (e, H(R, PK, m));
	match (TIDa, H(A_1, g));
	match (w, H(TIDa, pw));
	match (ma1, H(TIDa, TIDdb, w, Ta1, NA1, pk(A_1), QA, e, y, m))

2. A_1 sends a message to DB.	send_1(A_1,DB,{TIDa, DB, w, Ta1, NA1, y, ma1, QA}pk(DB));
3. DB receives the message.	recv_1(A_1,DB,{TIDa, DB, w, Ta1, NA1, y, ma1, QA}pk(DB));
4. DB sends a signature conversion request to the inter-chain notary NP_1	send_2(DB, NP_1,{DB, NP_1,Tdb1,NDB1,QA}pk(NP_1));
5. Notary NP_1 receives the message.	recv_2(DB, NP_1,{DB, NP_1,Tdb1,NDB1,QA}pk(NP_1));
6. Notary NP_1performs computations.	match(h,H(NP_1));
7. Notary NP_1 sends a response message to DB.	send_3(NP_1, DB, {NP_1, DB, Tnp1, NDB1, h, QDB}pk(DB));
8. DB receives the response message.	recv_3(NP_1, DB, {NP_1, DB, Tnp1, NDB1, h, QDB}pk(DB));
9. DB performs computations.	match(mdb,H(NP_1,DB, Z, Tdb2, NDB2, NA1, pk(DB), pk(NP_1), h, QDB));
10. DB sends a response message to A_1.	send_4(DB, A_1, {NP_1, DB, Z, Tdb2, NDB2, NA1, h, QDB, mdb}pk(A_1));
11.A_1 receives the response message.	recv_4(DB, A_1, {NP_1, DB, Z, Tdb2, NDB2, NA1, h, QDB, mdb}pk(A_1));

TABLE IV: Key source code for formal verification of security attributes

Security Attribute Modeling of A_1	Security Attribute Modeling of DB	Security Attribute Modeling of NP_1
claim(A_1,Secret,Ta1);	claim(DB,Secret,Tdb1);	claim(NP_1,Secret,Tnp1);
claim(A_1,Alive);	claim(DB,Secret,Tdb2);	claim(NP_1,Alive);
claim(A_1,Weakagree);	claim(DB,Alive);	claim(NP_1,Weakagree);
claim(A_1,Niagree);	claim(DB,Weakagree);	claim(NP_1,Niagree);
claim(A_1,Nisynch);	claim(DB,Niagree);	claim(NP_1,Nisynch);
	claim(DB,Nisynch);	

x," and "Alive(x)" implies that entity x is active or alive. Therefore, meaning that throughout the protocol execution, all entities remain active, without being unexpectedly interrupted or terminated.

(b) *Weakagree:*

$$\text{claim}(A_1, \text{forall x :forall y : NonRepudiation(x)\&\& NonRepudiation(y))}$$

"forall x, y" indicates "for all entities x and y", and "Weakagree(x, y)" denotes weak agreement between entities x and y.

(c) *Niagree:*

$$\text{claim}(A_1, \text{forall x :forall y : NonRepudiation(x)\&\& NonRepudiation(y))}$$

"NonRepudiation(x)" signifies that communication entity x cannot repudiate its participation in communication or negotiation results. Therefore, "forall x, y" implies that both entities x and y cannot repudiate their participation in communication or negotiation results. In the context of our protocol, this indicates that communication entity A_1 cannot repudiate its participation in communication or negotiation results.

(d) *Nisynch:*

$$\text{claim}(A_1, \text{forall x : forall y : Synchronized(x, y)} -> \text{NonRepudiation(x) \&\& NonRepudiation(y))}$$

In this expression, "forall" is a universal quantifier. "Synchronized(x, y)" indicates that communication entities x and y have achieved synchronization. "NonRepudiation(x)" signifies that communication entity x cannot repudiate its participation in communication or negotiation results. Therefore, for all communication entities x and y, if they achieve synchronization in the protocol using the predicates Synchronized, then they cannot repudiate their participation in communication or negotiation results using the predicates NonRepudiation. In the context of the proposed protocol, it implies that communication entity A_1 cannot repudiate its participation in communication or negotiation results if it achieves synchronization with other entities.

For the goals established based by BAN logic, goals G1 and G3 accomplish bidirectional authentication between A_1 and DB. Goals G2 and G4 establish consistency for the key K between A_1 and DB. By the *Scyther* tool, through the Niagree declaration between A_1 and DB, both parties cannot repudiate their participation in communication or negotiation results, meaning that they cannot repudiate their shared key K. It achieves the goals of G2 and G4. Similarly, by their Nisynch declarations, they indicate synchronization, accomplishing bidirectional authentication and meeting the goals of G1 and G3.

4) Formal Verification Results Analysis: Based on the above-mentioned modeling processes, with the help of *Scyther* formal verification platform, we can successfully verify the confidentiality, integrity and authentication of the MU-CCAS, and prove the availability and security of the MU-CCAS. The verification results are show in Figure 4.

The verification results show that the entities A_1, DB, and NP_1 in the MU-CCAS protocal meet the confidentiality, existence, weak consistency, non-monoinjective consistency, and non-injective synchrony. Therefore, the MU-CCAS meets the confidentiality and authentication to make the formal verification of the MU-CCAS successful. At the same time, our verification results also prove the reliability and correctness of logical inferences by BAN logic.

C. Security Attribute Analysis

The MU-CCAS is able to resist various network attacks to meet the security demand of cross-chain authentication. Here, we describe the analysis on security attributes.



Scyther results : verify

Claim				Status		Comments
KYRZ	A1	KYRZ,A11	Secret T1	Ok	Verified	No attacks.
		KYRZ,A12	Alive	Ok	Verified	No attacks.
		KYRZ,A13	Weakagree	Ok	Verified	No attacks.
		KYRZ,A14	Niagree	Ok	Verified	No attacks.
		KYRZ,A15	Nisynch	Ok	Verified	No attacks.
	DB	KYRZ,DB1	Secret T2	Ok	Verified	No attacks.
		KYRZ,DB2	Secret T4	Ok	Verified	No attacks.
		KYRZ,DB3	Alive	Ok	Verified	No attacks.
		KYRZ,DB4	Weakagree	Ok	Verified	No attacks.
		KYRZ,DB5	Niagree	Ok	Verified	No attacks.
		KYRZ,DB6	Nisynch	Ok	Verified	No attacks.
	NP1	KYRZ,NP11	Secret T3	Ok	Verified	No attacks.
		KYRZ,NP12	Alive	Ok	Verified	No attacks.
		KYRZ,NP13	Weakagree	Ok	Verified	No attacks.
		KYRZ,NP14	Niagree	Ok	Verified	No attacks.
		KYRZ,NP15	Nisynch	Ok	Verified	No attacks.

Done.

Fig. 4: *Scyther* tool verification results for MU-CCAS

1) Ability against Replay Attacks: In MU-CCAS, both A_1 and DB randomly select the random parameters for local time stamps to maintain message freshness, which can ensure the validity of the message during cross-chain authentication. If a malicious attacker intercepts a message and returns it to the cross-chain communication channel, the verification will eventually fail because the time stamp in the replay message and the random parameters for maintaining the freshness of the message are different from the original ones.

2) Ability against Substitution Attacks: By the proposed scheme, a secret value is selected to establish a temporary identity, and $w = H_2(TID_{A_1} || pw)$ is used to bind the password and temporary identity of the user. A malicious attacker cannot generate the temporary identity TID_{A_1} and w, nor can he obtain the $A_1's$ private key, so that he fails to form the message signature, making the final message m_1 failure to be authenticated.

3) Anonymous Tracing of Entity Identity: The MU-CCAS selects secret values to establish temporary identity to realize identity anonymity. If an illegal user A_m sent an illegal request message to DB, the DB submits the temporary identity TID_{A_m} of the illegal user to the KGC_B for authentication. The KGC_B checks the message. If the message is valid, the KGC_B queries the temporary identity TID_{A_m} in the local list, and calculates $TID_{A_m} = H_1(ID_{A_m} || g^{t_{A_m}})$ according to ID_{A_m} and $g^{t_{A_m}}$ to verify whether the calculation result is temporary identity TID_{A_m}. If the verification passes, it is ID_{A_m}, which sends the result from the KGC_B to the DB, so the MU-CCAS has a capacity for anonymous tracking.

4) Ability against Middleman Attacks: In A_1 to DB across-chain access, according to its temporary identity TID_U and password pw calculate secret parameter w, the local timestamp T_U, random parameter N_U, A_1 in the message package, including KGC_A signature issued by temporary identity, and in the process of communication the A_1 use DB public key encryption message, DB received the message can only use their private key to decrypt the message, decryption message after A_1 signature verification, ensure the authenticity of the identity, so as to resist the middleman attack.

5) Auditability : For different cross-chain behaviors, some of the notaries can be selected for supervision. The audit logs can record the cross-chain behaviors and the operations of the users. Within the blockchain, the node will record the user identity and access records to achieve auditability.

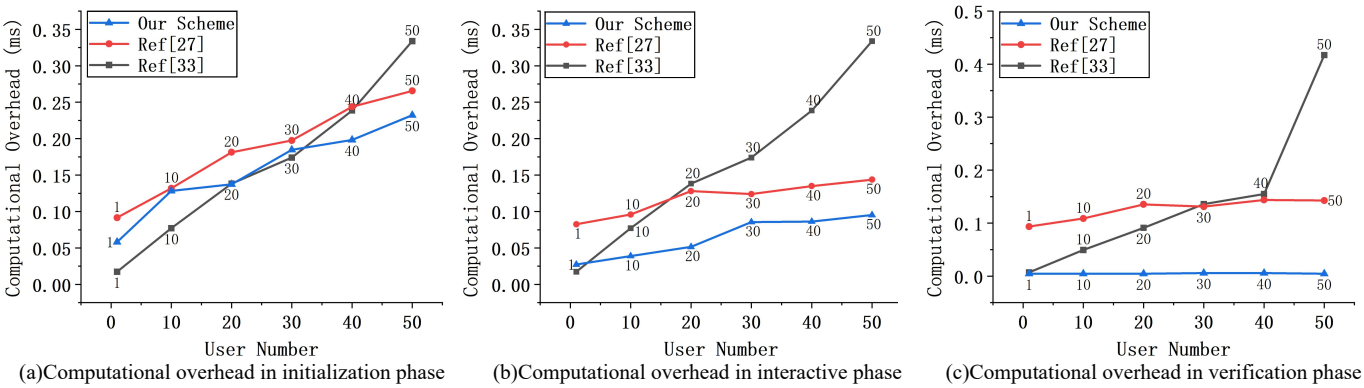
V. Performance Analysis

The performance testing focuses on the scenario that multiple users access the DB across blockchains. First, the computational overhead of the MU-PRS algorithm is evaluated using Golang language. Following it, the proposed MU-CCAS is deployed on Hyperledger Fabric for performance testing. Finally, a comparative analysis is conducted on the performance of cross-chain technology. The experiments have been conducted on a Dell machine with an Intel® Core™ i7-10700 CPU, 40GB of RAM. The host system runs Ubuntu 22.04.2 LTS, Go version 1.18.8, Hyperledger Fabric v2.2.0, Docker 20.10.21, docker-compose 1.29.2, node.js v12.22.9, and npm version 8.5.1.

A. Performance Analysis of the MU-PRS Algorithm

Golang language is employed to measure the computational overhead of the MU-PRS algorithm during the key generation stage, the generation and verification of signature stage. 50 measurements are conducted for each scenario with the users counting of 1, 10, 100, and 1000, and the results represent the averages of these measurements. The results indicate that as the number of users increases, the computational overhead for generating the re-signature key R_{AB}, generating the re-signature δ_{DB}, and verifying the re-signature δ_{DB} stabilize and remain relatively low. Therefore, the MU-PRS algorithm exhibits favorable performance. The test results are presented in Table V. The unit of data in the table is seconds. It's worth noting that the average execution time for bilinear mapping is $1.29E^{-}4$ milliseconds, and for exponentiation, it is 1.25 milliseconds.

In the key generation phase, the MU-PRS algorithm exhibits relatively low computational overhead for generating a single user's key. However, as the number of users doubles, the computational overhead for generating key pairs for each user also double because each user needs to generate his own public-private key pair. When it comes to generating aggregate public and private keys for the "i" users, the computational overhead increase linearly with the number of users. Nevertheless, even when the user count reaches 1000, the computational overhead remain relatively low. The computational overhead



(a)Computational overhead in initialization phase (b)Computational overhead in interactive phase (c)Computational overhead in verification phase

Fig. 5: The comparison results of the computational overhead

TABLE V: Computational overhead of the MU-PRS algorithm (Unit: millisecond)

Number of Users A_1	Key generation stage			Generation and verification of signature stage		
	pk_{ai} and sk_{ai} Gen	PK and SK Gen	Re-sig Key R_{AB} Gen	Agg Sig δ_A Gen	Re-sig δ_{DB} Gen	Sig Verification
1	4.25E-03	7.39E-04	1.15E-02	8.11E-03	1.71E-03	4.35E-03
10	2.82E-02	5.46E-03	1.40E-02	2.25E-02	2.03E-03	4.88E-03
100	2.69E-01	5.70E-02	2.14E-02	1.54E-01	2.26E-03	5.28E-03
10000	2.87	5.70E-01	2.39E-02	1.70	3.81E-03	7.65E-03

for generating the R_{AB} are stable because we aggregate the keys of "i" users aggregate before generating the R_{AB}.

In the generation and verification of signature stage, the time it taken for the MU-PRS algorithm to generate the aggregate signature δ_A for each user A_i is linearly related to the number of users and constitutes a significant portion of the algorithm's total time. Generating the aggregate signature δ_A involves each user A_i generating his own signature for the message, and then aggregating the δ_A of the multiple users. Therefore, the time required for generating the δ_A increases with the number of users. Generating a re-signature δ_{DB} involves performing calculations with the R_{AB} and the δ_A. However, the MU-PRS algorithm doesn't require one-to-one re-signature generation, so the impact of the number of users on the generation of the δ_{DB} is relatively small, and the computational overhead is low. In the signature verification phase, the DB uses its private key to verify the signature transformed by the inter-chain notary. Signature verification only needs to be performed once, and there's no need to verify the signature for each user individually. Therefore, the computational overhead is relatively low and stable.

B. Performance Analysis of the MU-CCAS

To evaluate the performance of MU-CCAS, we segmented it into three distinct phases: initialization, interaction, and verification, conducting comparisons with references [27] and [33]. Computational overhead is quantified as the cumulative computational efforts of all entities within each stage. reference [27] represents a significant contribution to enabling heterogeneous cross-domain authentication within a blockchain environment, while reference [33] is a pivotal study that employs the proxy re-signature algorithm to facilitate cross-domain authentication. For an equitable calculation of computational overhead, we adapted the trust domains from these cross-domain schemes to a blockchain setting, designated cloud servers or blockchain nodes as inter-chain notaries, and subjected all schemes to experimental evaluation in a cross-chain context. Excluding factors like blockchain forwarding, queries, and delays, our analysis focuses solely on the direct interactions among user A_i, database DB, and inter-chain notary NP_i, averaging the computational overhead across 50 one-way cross-domain verifications.

Figure 5 (a) illustrates the comparison of computational overhead during the system initialization phase. Due to the necessity of generating an identity, along with public and private keys for each user, the computational overhead for all three schemes escalates as the number of users increases. At this stage, MU-CCAS assigns a temporary identity to users, whereas reference [27] creates temporary identities during the interaction phase. Nonetheless, MU-CCAS generally exhibits lower computational overhead. However, as the total user count rises, the computational overhead of reference [33] significantly increases, surpassing that of MU-CCAS when the user count exceeds 33.

The interaction phase commences with User A_1 issuing an access request and concludes prior to DB's final identity verification for A_1. This phase's computational overhead comparison is illustrated in Figure 5 (b). Here, MU-CCAS maintains a relatively low computational overhead. When user numbers hit 50, the computational overhead for [27] is approximately 1.5 times that of MU-CCAS, and for reference [33], about 3.5 times. This discrepancy arises because each user must sign their message before dispatching a cross-chain access request, thus scaling the signature generation's computational overhead with user increments. However, MU-CCAS integrates additional steps for computing aggregated signatures at a reduced overhead. Once user signatures are combined, MU-CCAS necessitates a single interaction with DB and NP_i, irrespective of user volume. In contrast, the interaction frequency and, consequently, the computational

TABLE VI: Performance analysis of nodes deploying the MU-CCAS in the blockchain environment

User# (A_i)	Throughput (TPS)	Avg. Delay (s)	Avg. Memory Consumption	
			$PeerA_1$(MB)	$Peer_{DB}$(MB)
1	226.4	0.01	9.68	10.5
10	94.3	0.03	12.8	15.6
100	9.2	0.27	38.3	29.1
1000	1.2	2.71	251	233

overhead in references [27], [33] ascend with the user count.

The verification phase involves verifying A_1's identity. As Figure 5 (c) shows, MU-CCAS's authentication computational overhead is less than that observed in references [27], [33], and its efficiency becomes increasingly evident as the user base expands. This efficiency stems from MU-CCAS verifying a signature that has been transformed from an aggregated signature, enabling a single verification step regardless of the number of users. Meanwhile, references [27], [33] necessitate individual identity verification for each user.

The performance of the MU-CCAS is tested in the blockchain environment. We test 100 times with the number of users to be 1, 10, 100, and 1000, respectively. The final results have been obtained by calculating the average values. The blockchain environment used is the Hyperledger Fabric v2.2.0 with chain code written in Golang language, and the testing scripts written in JavaScript.

As shown in Table VI, the increase in the number of users A_i leads to a linear growth in both average memory consumption on $PeerA_1$ (Within the $Blockchain_A$) and $PeerB_1$ (Within the $Blockchain_B$). This linear relationship also affects the average latency, resulting in reduced throughput. The reason behind this behavior is that resource requirements and transaction execution time are directly proportional to the number of users. However, memory consumption and latency are significantly influenced by activities like state database modifications and block generation, which consume a substantial portion of total memory. This proportion increases as the number of users decreases. Additionally, as the number of users grows, transaction execution time rises linearly, primarily due to processing data for each individual user. Notably, with a small number of users, processing user data is swift, and most latency results from other process components.

C. Performance Analysis Based on Cross-Chain Technology

Due to the variations of performance among different cross-chain technologies, their implementation difficulty also varies across different application scenarios. In this section, the proposed approach is analyzed in terms of scalability, verifiability, application scope, and implementation complexity and it is compared with other authentication schemes.

When the proposed scheme is compared with other schemes on the dimension of cross-chain technology, the results in Table VII indicate that the proposed approach, as well as the solutions in [21], [41], [42], are suitable for a wide range of application scenarios. However, the proposed approach has relatively lower implementation difficulty and higher scalability. In comparison with the solutions in [20], [9], [43], the implementation difficulty of the proposed approach remains relatively low, and its application scope is more extensive with strong scalability. Therefore, compared with other existing schemes, the MU-CCAS has a wider range of application scenarios, less difficult to implement, more practical, and more feasible for cross-chain access to resource circulation.

TABLE VII: Performance analysis and comparison of authentication scheme based on cross-chain technology

Method	Cross-Chain Technology	Scalability	Application Scope	Impl. Difficulty
[20]	Hash Locking + Notary	High	Limited	Easy
[21]	Sidechain	Low	Wide	Difficult
[41]	Relay	Low	Wide	Moderate
[42]	Relay + Hash Locking	Low	Wide	Moderate
[9]	Hash Locking	Low	Limited	Easy
[43]	Notary	Moderate	Limited	Easy
Ours	Notary	High	Wide	Easy

VI. Related Work

A. The Cross-system Authentication Based on Blockchain

Indeed, many researchers consider the blockchain as a distributed and trusted entity that can be leveraged for authentication across different trust systems. This is achieved by establishing trust roots [13], [14] and employing signature validation mechanisms [15] , [16], [17], [18]. Le et al. [14] classified devices within institutions based on roles and realized mutual authentication of mobile medical devices between different healthcare institutions using a trust list. Dong et al. [15] used threshold-shared multiple signatures to build a collaborative domain identity alliance, implementing a blockchain-based cross-domain authentication scheme for drones. Zhang et al. [16] proposed a comprehensive identity authentication scheme that allows different domains to adopt various cryptographic systems. Participants from different domains can directly invoke chaincode for digital signatures or hash calculations. Wang et al. [17] combined accumulator knowledge and digital signature schemes, formulating the authentication problem as a signature propagation issue. Yang et al. [13] developed two distributed authentication protocols utilizing the avatar's identity model and chameleon collision signatures, enabling real-time authentication of virtual characters within the metaverse. Wang et al. [18] integrated the lightweight Boneh-Lynn-Shacham signature with a blockchain-based group signature to facilitate multi-user and multi-task batch anonymous authentication. By calculating the authentication information from signers to verifiers and proving its legitimacy, they resolved cross-domain authentication issues. While the schemes [13], [14], [15], [16], [17], [18] achieve decentralized identity authentication. However, they are primarily implemented within a single blockchain and do not consider the issue of resource circulation between different blockchain systems in multi-chain scenarios.

B. Cross-Chain Authentication

Many researchers have combined various cross-chain technologies to develop cross-chain smart contracts, aiming to achieve transaction authentication among different blockchain networks. Zhang et al. [2] developed a dual-chain system comprising a Digital Asset Chain and a Payment Chain, aimed

at facilitating secure transactions and payments through a cross-chain mechanism. Lys et al. [42] improved hash-locked cross-chain transactions using relays and adapters, enabling automated user operations and contract auditing. The solutions [2], [42] utilize sidechain or relay mechanisms, offering strong scalability and broad application scenarios, yet they may involve high implementation complexity. The solutions [3], [9] focused on asset transfer verification using hash locks and time locks to ensure asset security. While their solution exhibits high security, its applicability is limited. Borkowski et al. [43] introduced the DeXTT cross-chain transfer protocol based on a notary mechanism, allowing decentralized token transfers across multiple blockchains. The solutions [2], [42], [3], [9], [43] ensure atomicity and consistency in cross-chain transactions. However, they are mainly designed for certifying digital asset transfers and exchanges, making it challenging to address the issue of authentication in cross-chain resource circulation.

C. Proxy Re-Signature

PRS holds significant advantages in the context of cross-chain authentication for resource circulation. It enables the conversion of user signatures across different systems, thus facilitating the authentication of user identities within distinct blockchain networks. The concept of proxy re-signature was initially introduced by Blaze et al. [44]. Currently, many schemes for cross-system authentication use PKI-based PRS technology [26], [45], [46]. However, these solutions rely on CAs as certificate authorities, providing trust roots for all PKI certificates to facilitate user identity verification. This approach faces issues such as complex certificate management and difficulties in certificate revocation, and CAs are susceptible to potential attacks [47]. To address the issues with PKI, many scholars have proposed Identity-Based Proxy Re-Signature (IB-PRS) algorithms [31], [48]. These algorithms replace certificates with user identities, resolving the problems associated with certificate-based PRS algorithms. However, IB-PRS algorithms generate private keys entirely by the KGC, raising concerns about key custody and other security issues. On the other hand, certificateless proxy re-signature (CL-PRS) algorithms [32] eliminate the need for certificates. Private keys are jointly generated by the user and the KGC, effectively addressing the certificate management issues present in PKI-based PRS algorithms and the key custody issues in IB-PRS algorithms .The CL-PRS has gained favor among many researchers [33].

VII. Discussion and future work

The paper implements a Multi-User Cross-Chain Authentication Scheme (MU-CCAS). Its effectiveness relies on the assumption that blockchain systems use certificateless cryptographic and identity-based cryptography algorithms, limiting its applicability to blockchains with other cryptographic algorithms. The MU-NC mechanism involves a three-party interaction (users, databases, and notaries) to generate re-signing keys, reducing computational costs for cross-chain access, but requires all parties to be online simultaneously. Two critical factors affecting experiments are user device performance, suitable for up to 1,000 users, and data size in data sharing. Notably, data sharing occurs post-session channel establishment, and the study does not address the size of shared data.

Future work involves leveraging the multi-notary mechanism for cross-chain heterogeneous authentication and designing traceable and auditable notary mechanisms. Computational overhead reduction focuses on enabling users to independently generate re-signing keys, enhancing authentication efficiency. Additionally, outsourcing methods for aggregate signature generation aim to address users' devices' limitations in handling complex computational tasks.

VIII. Conclusion

This paper's principal contribution is the introduction of the MU-PRS algorithm and MN-SC mechanism, designed to aggregate the signatures of multiple users before conversion. Building upon the MU-PRS algorithm and MN-SC mechanism, we've developed the MU-CCAS, a solution that facilitates cross-chain authentication for multiple users while championing decentralization. By safeguarding both data and user identities within the system, MU-CCAS elevates the overall reliability and security of our solution.

Acknowledgments

This work was supported by National Key Research and Development Program of China (2021YFF1201102).

References

[1] M. Dotan, Y.-A. Pignolet, S. Schmid, S. Tochner, and A. Zohar, "Survey on blockchain networking: Context, state-of-the-art, challenges," *ACM Computing Surveys (CSUR)*, vol. 54, no. 5, pp. 1–34, 2021.

[2] P. Zhang, X. Hua, and H. Zhu, "Cross-chain digital asset system for secure trading and payment," *IEEE Transactions on Computational Social Systems*, vol. 11, no. 2, pp. 1654–1666, 2024.

[3] Y. Chen, J.-N. Liu, A. Yang, J. Weng, M.-R. Chen, Z. Liu, and M. Li, "Pacdam: Privacy-preserving and adaptive cross-chain digital asset marketplace," *IEEE Internet of Things Journal*, vol. 11, no. 8, pp. 13 424–13 436, 2024.

[4] R. Nowrozy, K. Ahmed, A. Kayes, H. Wang, and T. R. McIntosh, "Privacy preservation of electronic health records in the modern era: A systematic survey," *ACM Computing Surveys*, vol. 56, no. 8, pp. 1–37, 2024.

[5] L. Xue, H. Huang, F. Xiao, and W. Wang, "A cross-domain authentication scheme based on cooperative blockchains functioning with revocation for medical consortiums," *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2409–2420, 2022.

[6] R. Belchior, A. Vasconcelos, S. Guerreiro, and M. Correia, "A survey on blockchain interoperability: Past, present, and future trends," *Acm Computing Surveys (CSUR)*, vol. 54, no. 8, pp. 1–41, 2021.

[7] A. Lohachab, S. Garg, B. Kang, M. B. Amin, J. Lee, S. Chen, and X. Xu, "Towards interconnected blockchains: A comprehensive review of the role of interoperability among disparate blockchains," *ACM Computing Surveys (CSUR)*, vol. 54, no. 7, pp. 1–39, 2021.

[8] H. Jin and J. Xiao, "Towards trustworthy blockchain systems in the era of "internet of value": development, challenges, and future trends," *Science China Information Sciences*, vol. 65, pp. 1–11, 2022.

[9] B. Pillai, K. Biswas, Z. Hóu, and V. Muthukkumarasamy, "The burn-to-claim cross-blockchain asset transfer protocol," in *2020 25th International Conference on Engineering of Complex Computer Systems (ICECCS)*. IEEE, 2020, pp. 119–124.

[10] L. Hong, A. Jain, S. Pankanti, and R. Bolle, "Identity authentication using fingerprints," in *Audio-and Video-based Biometric Person Authentication: First International Conference, AVBPA'97 Crans-Montana, Switzerland, March 12–14, 1997 Proceedings 1*. Springer, 1997, pp. 103–110.

[11] W. Wang, N. Hu, and X. Liu, "Blockcam: a blockchain-based cross-domain authentication model," in *2018 IEEE Third International Conference on Data Science in Cyberspace (DSC)*. IEEE, 2018, pp. 896–901.

[12] J. Chen, Z. Zhan, K. He, R. Du, D. Wang, and F. Liu, "Xauth: Efficient privacy-preserving cross-domain authentication," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3301–3311, 2021.

[13] K. Yang, Z. Zhang, T. Youliang, and J. Ma, "A secure authentication framework to guarantee the traceability of avatars in metaverse," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 3817–3832, 2023.

[14] L. Lai, T. Zhou, Z. Cai, J. Yu, H. Bai, and J. Cui, "Leveraging blockchain for cross-institution data sharing and authentication in mobile healthcare," in *2021 17th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE, 2021, pp. 311–318.

[15] S. Dong, H. Yang, J. Yuan, L. Jiao, A. Yu, and J. Zhang, "Blockchain-based cross-domain authentication strategy for trusted access to mobile devices in the iot," in *2020 International Wireless Communications and Mobile Computing (IWCMC)*. IEEE, 2020, pp. 1610–1612.

[16] H. Zhang, X. Chen, X. Lan, H. Jin, and Q. Cao, "Btcas: A blockchain-based thoroughly cross-domain authentication scheme," *Journal of Information Security and Applications*, vol. 55, p. 102538, 2020.

[17] L. Wang, Y. Tian, and D. Zhang, "Toward cross-domain dynamic accumulator authentication based on blockchain in internet of things," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 4, pp. 2858–2867, 2021.

[18] C. Wang, Y. Zhang, Q. Zhang, X. Xu, W. Chen, and H. Li, "Secas: Secure and efficient cross-domain authentication scheme based on blockchain for space ttc networks," *IEEE Internet of Things Journal*, pp. 1–1, 2024.

[19] S. Valtolina, L. Ferrari, and M. Mesiti, "Ontology-based consistent specification of sensor data acquisition plans in cross-domain iot platforms," *IEEE Access*, vol. 7, pp. 176 141–176 169, 2019.

[20] Y. Sun, L. Yi, L. Duan, and W. Wang, "A decentralized cross-chain service protocol based on notary schemes and hash-locking," in *2022 IEEE International Conference on Services Computing (SCC)*. IEEE, 2022, pp. 152–157.

[21] A. Garoffolo, D. Kaidalov, and R. Oliynykov, "Zendoo: A zk-snark verifiable cross-chain transfer protocol enabling decoupled and decentralized sidechains," in *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2020, pp. 1257–1262.

[22] Z. Yin, B. Zhang, J. Xu, K. Lu, and K. Ren, "Bool network: An open, distributed, secure cross-chain notary platform," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3465–3478, 2022.

[23] Y. Ding, Y. Zhang, B. Qin, Q. Wang, Z. Yang, and W. Shi, "A scalable cross-chain access control and identity authentication scheme," *Sensors*, vol. 23, no. 4, p. 2000, 2023.

[24] Y. He, C. Zhang, B. Wu, Y. Yang, K. Xiao, and H. Li, "A cross-chain trusted reputation scheme for a shared charging platform based on blockchain," *IEEE Internet of Things Journal*, vol. 9, no. 11, pp. 7989–8000, 2021.

[25] S. Shao, F. Chen, X. Xiao, W. Gu, Y. Lu, S. Wang, W. Tang, S. Liu, F. Wu, J. He *et al.*, "Ibe-bciot: An ibe based cross-chain communication mechanism of blockchain in iot," *World Wide Web*, vol. 24, no. 5, pp. 1665–1690, 2021.

[26] X. Yang, F. An, and P. Yang, "Cross-domain authentication scheme based on proxy re-signature in cloud environment," *Chinese Journal of Computers*, vol. 42, no. 4, pp. 756–771, 2019.

[27] F. Tong, X. Chen, K. Wang, and Y. Zhang, "Ccap: A complete cross-domain authentication based on blockchain for internet of things," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3789–3800, 2022.

[28] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in *2007 IEEE symposium on security and privacy (SP'07)*. IEEE, 2007, pp. 321–334.

[29] A. Sahai and B. Waters, "Fuzzy identity-based encryption," in *Advances in Cryptology–EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005. Proceedings 24*. Springer, 2005, pp. 457–473.

[30] M. Burrows, M. Abadi, and R. Needham, "A logic of authentication," *ACM Transactions on Computer Systems (TOCS)*, vol. 8, no. 1, pp. 18–36, 1990.

[31] J. Zhang, "Improvement of id-based proxy re-signature scheme with pairing-free," *Wireless Networks*, vol. 25, pp. 4319–4329, 2019.

[32] Y. Zhou, S. Dong, and Y. Yang, "A unidirectional certificateless proxy re-signature scheme based on lattice," *Transactions on Emerging Telecommunications Technologies*, vol. 33, no. 4, p. e4412, 2022.

[33] Y. Chen, W. Xu, L. Peng, and H. Zhang, "Light-weight and privacy-preserving authentication protocol for mobile payments in the context of iot," *IEEE Access*, vol. 7, pp. 15 210–15 221, 2019.

[34] R. Qiao, X.-Y. Luo, S.-F. Zhu, A.-D. Liu, X.-Q. Yan, and Q.-X. Wang, "Dynamic autonomous cross consortium chain mechanism in e-healthcare," *IEEE journal of biomedical and health informatics*, vol. 24, no. 8, pp. 2157–2168, 2020.

[35] D. Xie, J. Yang, B. Wu, W. Bian, F. Chen, and T. Wang, "An effectively applicable to resource constrained devices and semi-trusted servers authenticated key agreement scheme," *IEEE Transactions on Information Forensics and Security*, 2024.

[36] C. Lai, Y. Ma, R. Lu, Y. Zhang, and D. Zheng, "A novel authentication scheme supporting multiple user access for 5g and beyond," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 4, pp. 2970–2987, 2022.

[37] Y. Yang, J. Cao, R. Ma, L. Cheng, L. Chen, B. Niu, and H. Li, "Fhap: Fast handover authentication protocol for high-speed mobile terminals in 5g satellite–terrestrial-integrated networks," *IEEE Internet of Things Journal*, vol. 10, no. 15, pp. 13 959–13 973, 2023.

[38] Z. Shang, M. Ma, and X. Li, "A secure group-oriented device-to-device authentication protocol for 5g wireless networks," *IEEE Transactions on Wireless Communications*, vol. 19, no. 11, pp. 7021–7032, 2020.

[39] C. J. Cremers, "The scyther tool: Verification, falsification, and analysis of security protocols: Tool paper," in *International conference on computer aided verification*. Springer, 2008, pp. 414–418.

[40] D. X. Song, "Athena: a new efficient automatic checker for security protocol analysis," in *Proceedings of the 12th IEEE Computer Security Foundations Workshop*. IEEE, 1999, pp. 192–202.

[41] Z. Liu, Y. Xiang, J. Shi, P. Gao, H. Wang, X. Xiao, B. Wen, and Y.-C. Hu, "Hyperservice: Interoperability and programmability across heterogeneous blockchains," in *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, 2019, pp. 549–566.

[42] L. Lys, A. Micoulet, and M. Potop-Butucaru, "Atomic cross chain swaps via relays and adapters," in *Proceedings of the 3rd Workshop on Cryptocurrencies and Blockchains for Distributed Systems*, 2020, pp. 59–64.

[43] M. Borkowski, M. Sigwart, P. Frauenthaler, T. Hukkinen, and S. Schulte, "Dextt: Deterministic cross-blockchain token transfers," *IEEE access*, vol. 7, pp. 111 030–111 042, 2019.

[44] M. Blaze, G. Bleumer, and M. Strauss, "Divertible protocols and atomic proxy cryptography," in *International conference on the theory and applications of cryptographic techniques*. Springer, 1998, pp. 127–144.

[45] M. Song, Z. Hua, Y. Zheng, H. Huang, and X. Jia, "Blockchain-based deduplication and integrity auditing over encrypted cloud storage," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4928–4945, 2023.

[46] R. Rabaninejad, M. A. Attari, M. R. Asaar, and M. R. Aref, "A lightweight auditing service for shared data with secure user revocation in cloud storage," *IEEE Transactions on Services Computing*, vol. 15, no. 1, pp. 1–15, 2022.

[47] P. Szalachowski, "Password-authenticated decentralized identities," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4801–4810, 2021.

[48] J. Shao, G. Wei, Y. Ling, and M. Xie, "Unidirectional identity-based proxy re-signature," in *2011 IEEE International Conference on Communications (ICC)*. IEEE, 2011, pp. 1–5.